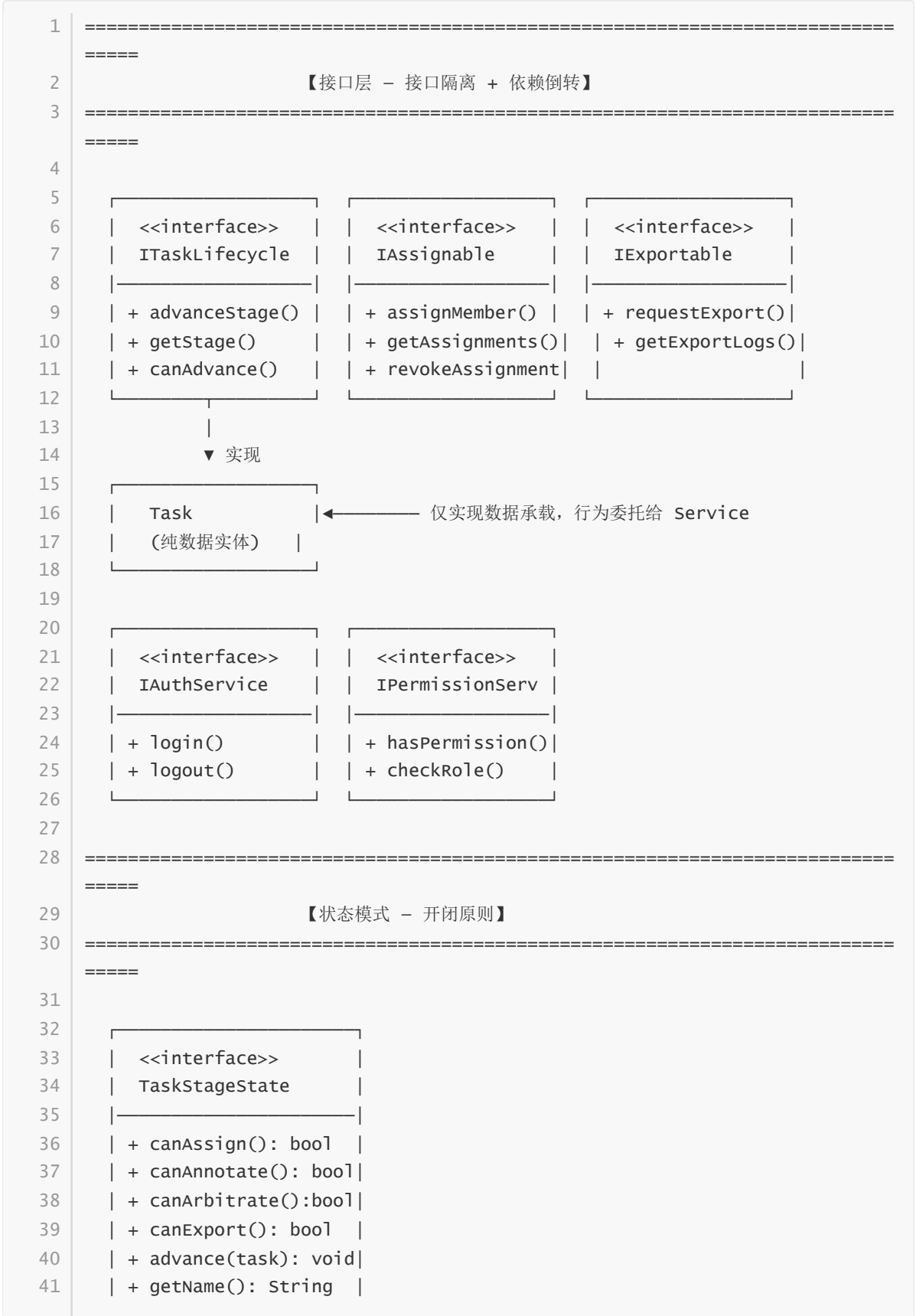


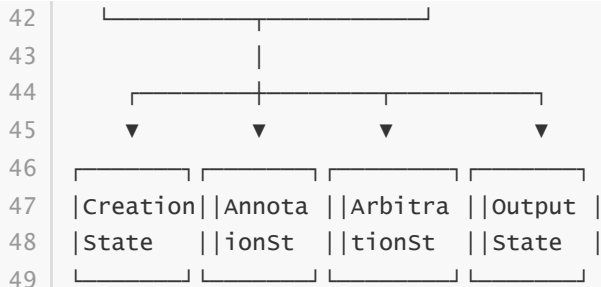
详细设计文档

团队：第44组

日期：5月16日

一、类图设计





51 Task 持有 TaskStageState 引用（组合），行为委托给当前状态对象

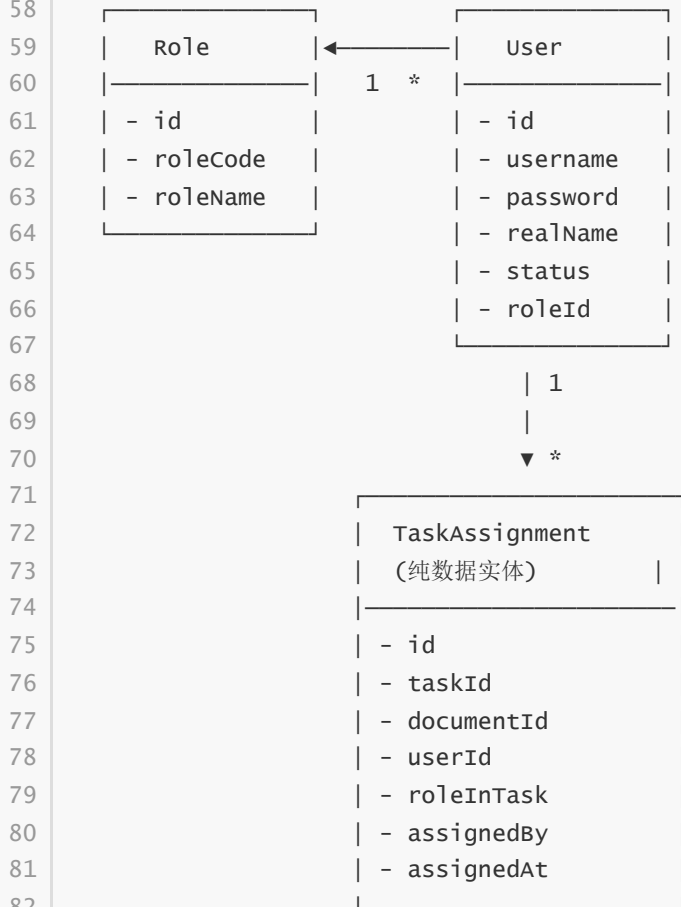
53 =====

54 【持久化实体层 - 单一职责（纯数据）】

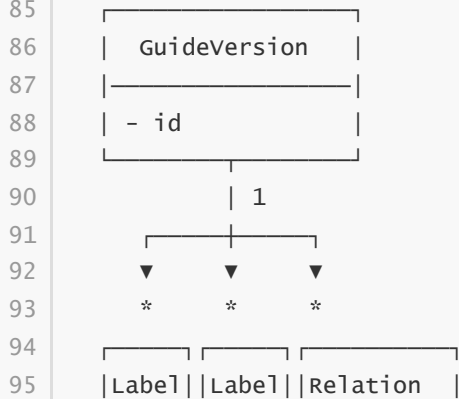
55 =====

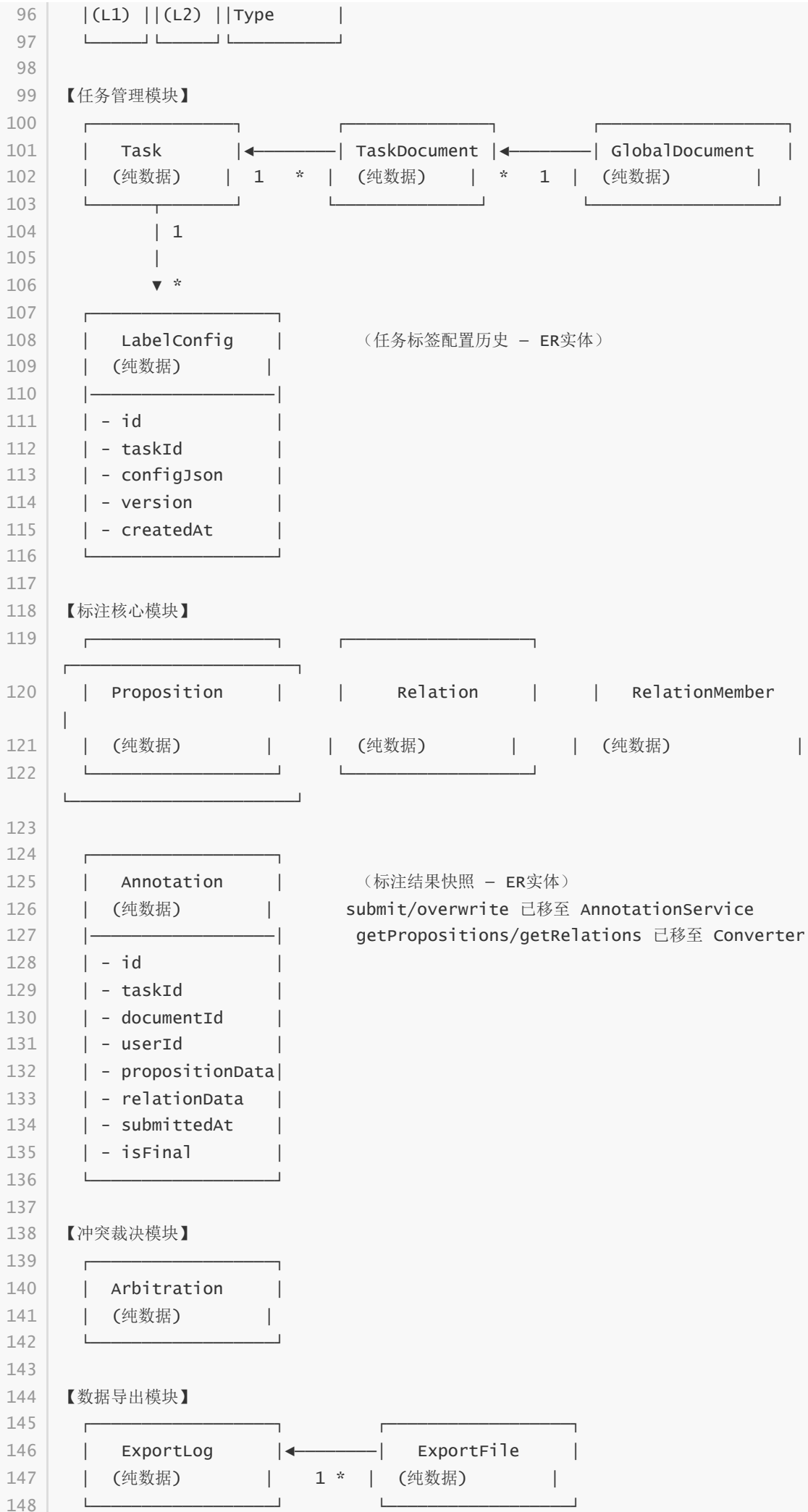
56 =====

57 【用户权限模块】



84 【配置中心模块】

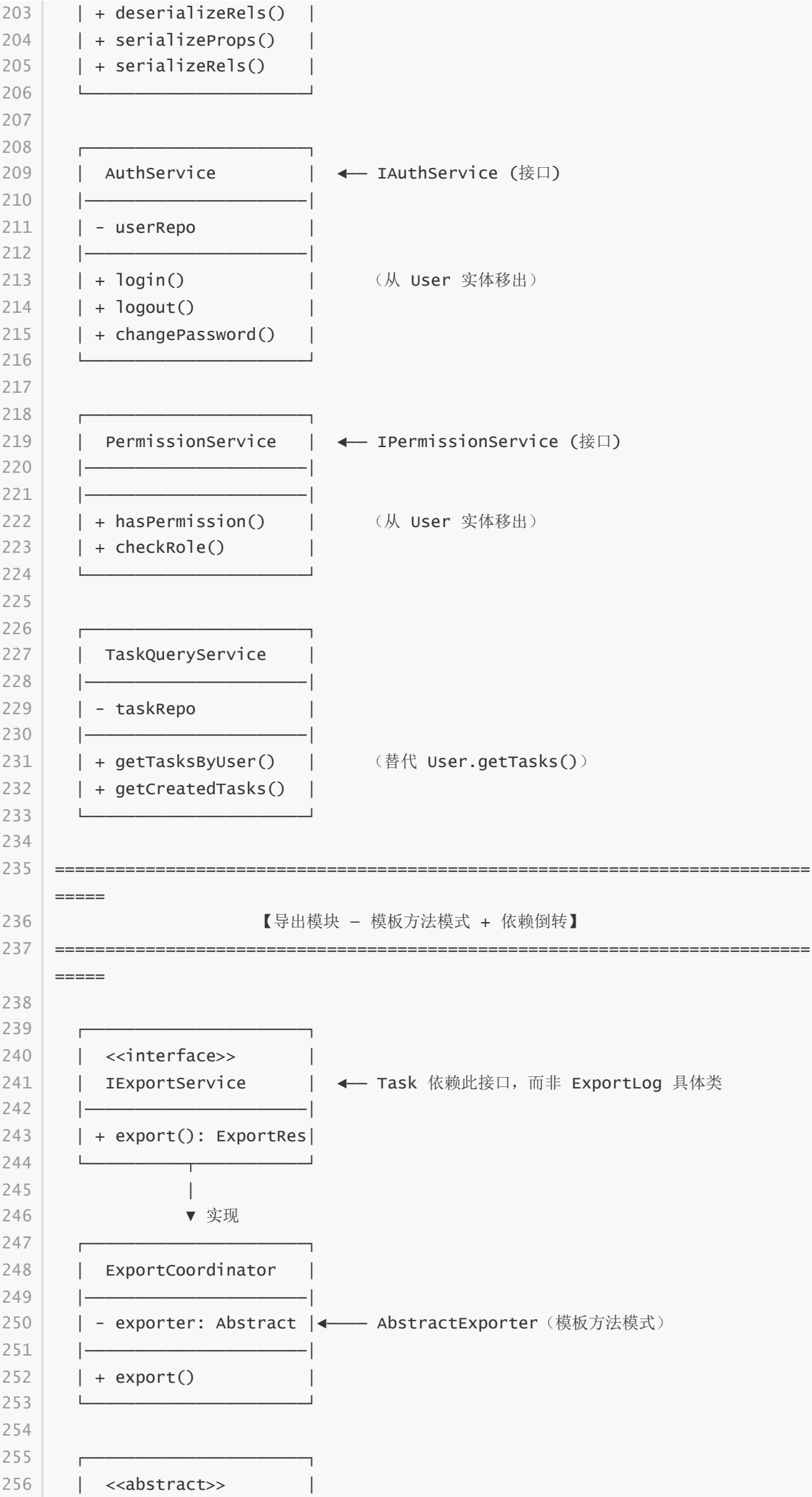


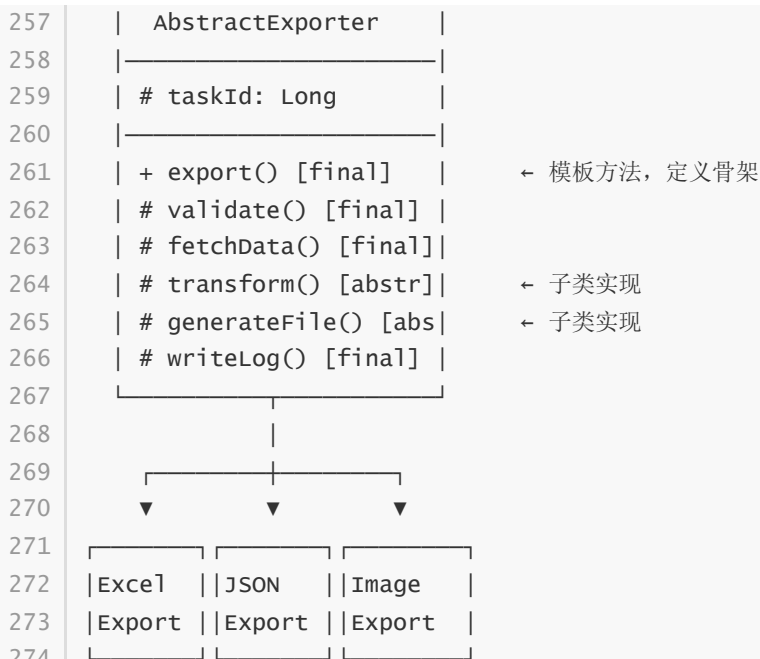


```

149
150 =====
151
152 =====
153
154
155 | TaskLifecycleService | ← ITaskLifecycle (接口)
156 |-----|
157 | - taskRepository |
158 | - state: TaskStageState | ← TaskStageState (状态模式)
159 |-----|
160 | + advanceStage() |
161 | + canAdvance() |
162 | + getStage() |
163 |-----|
164
165
166 | TaskAssignmentServ | ← IAssignable (接口)
167 |-----|
168 | - assignmentRepo |
169 |-----|
170 | + assignMember() | 统一入口: assignMember(uid, docId, roleType)
171 | + getAssignments() | 替代原 assignAnnotator/assignArbitrator
172 | + revokeAssignment() |
173 |-----|
174
175
176 | LabelConfigService |
177 |-----|
178 | - configRepo |
179 |-----|
180 | + saveConfig() |
181 | + getHistory() |
182 | + getAtVersion() |
183 | + rollbackTo() |
184 |-----|
185
186
187 | AnnotationService |
188 |-----|
189 | - annotationRepo |
190 | - converter | ← AnnotationDataConverter
191 |-----|
192 | + submit() | (从 Annotation 实体移出)
193 | + overwrite() |
194 | + getPropositions() | (委托给 Converter)
195 | + getRelations() |
196 |-----|
197
198
199 | AnnotationDataConver |
200 |-----|
201 |-----|
202 | + deserializeProps() | (从 Annotation 实体移出)

```





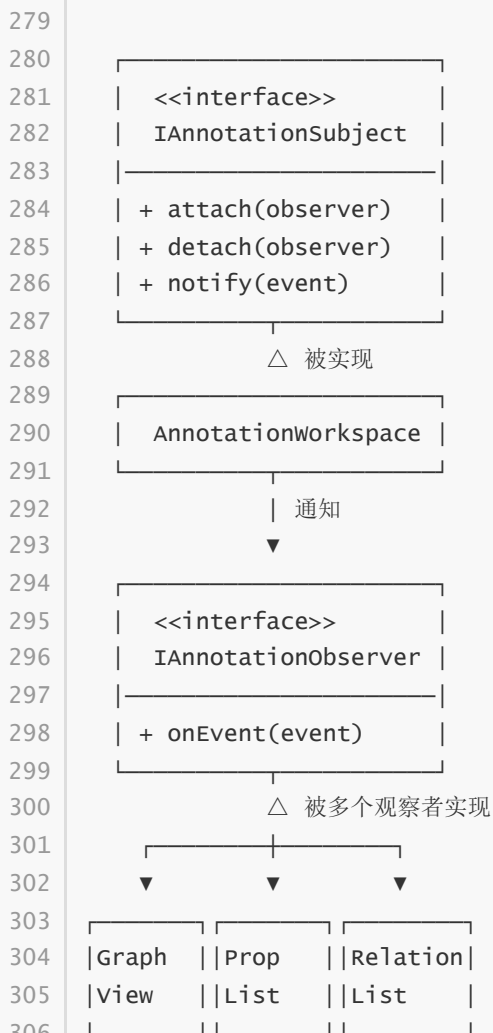
=====

=====

277 【观察者模式 - 解耦标注与图示】

278 =====

=====



=====

=====

309 【策略模式 - 文档解析】

序号	ER 图实体	类图对应类	状态	职责定位
6	关系类型表	RelationType	✓	纯数据实体
7	任务表	Task	✓	纯数据实体，行为委托给 Service 和 State
8	全局文件库	GlobalDocument	✓	纯数据实体
9	任务文档表	TaskDocument	✓	纯数据实体
10	任务分配表	TaskAssignment	✓	纯数据实体
11	命题表	Proposition	✓	纯数据实体
12	关系表	Relation	✓	纯数据实体
13	关系成员明细表	RelationMember	✓	纯数据实体
14	标注结果快照表	Annotation	✓	纯数据实体，submit/overwrite 已移至 Service
15	裁决结果表	Arbitration	✓	纯数据实体
16	导出日志表	ExportLog	✓	纯数据实体
17	导出文件明细表	ExportFile	✓	纯数据实体
18	任务标签配置历史表	LabelConfig	✓	纯数据实体，已从 Task 的 JSON 字段提取为独立类

1.3 各类详细定义

1.3.0 接口层（新增 — 接口隔离 + 依赖倒转）

ITaskLifecycle（任务生命周期接口）

方法签名	说明
+ advanceStage(taskId): void	推进任务到下一阶段
+ canAdvance(taskId): boolean	检查是否满足推进条件
+ getCurrentStage(taskId): TaskStage	获取当前阶段

接口隔离：仅包含生命周期相关方法，导出/分配等不相关方法不在此接口。

IAssignable（任务分配接口）

方法签名	说明
+ assignMember(taskId, documentId, userId, roleType): TaskAssignment	统一分配入口，roleType 为枚举
+ getAssignments(taskId): List<TaskAssignment>	获取任务分配列表
+ revokeAssignment(assignmentId): void	撤销分配

开闭原则：新增角色（如"审核员"）只需扩展 TaskRole 枚举，无需新增接口方法。

IExportable（导出接口）

方法签名	说明
+ requestExport(taskId, format, requestedBy): ExportResult	发起导出，返回 ExportResult 抽象结果
+ getExportLogs(taskId): List<ExportLog>	获取导出历史

依赖倒转：Task 依赖 IExportable 接口，而非 ExportLog 具体类。ExportResult 是包含 ExportLog 的封装对象。

IAuthService（认证服务接口）

方法签名	说明
+ login(username, password): AuthToken	用户登录验证
+ logout(token): void	退出登录
+ changePassword(userId, oldPwd, newPwd): boolean	修改密码

单一职责：从 User 实体中剥离，独立为横切关注点服务。

IPermissionService（权限服务接口）

方法签名	说明
+ hasPermission(userId, permissionCode): boolean	权限校验
+ checkRole(userId, roleCode): boolean	角色校验

单一职责：从 User 实体中剥离。

IAnnotationSubject（标注主题接口 — 观察者模式）

方法签名	说明
+ attach(observer: IAnnotationObserver): void	注册观察者
+ detach(observer: IAnnotationObserver): void	移除观察者
+ notify(event: AnnotationEvent): void	通知所有观察者

IAnnotationObserver（标注观察者接口）

方法签名	说明
+ onAnnotationEvent(event: AnnotationEvent): void	接收标注变更事件

IDocumentParser（文档解析接口 — 策略模式）

方法签名	说明
+ parse(file): ParsedDocument	解析文件提取文本

接口隔离：仅保留解析核心方法，extractReasonSection() 分离到 IReasonExtractable。

IReasonExtractable（裁判理由提取接口）

方法签名	说明
+ extractReasonSection(text): String	提取裁判理由/本院认为部分

接口隔离：预防 DocumentParser 变胖。PDF/Word 解析器可同时实现 IDocumentParser 和 IReasonExtractable。

IExportService（导出服务接口 — 依赖倒转）

方法签名	说明
+ export(taskId, format, requestedBy): ExportResult	执行导出

依赖倒转：Task 通过此接口间接使用导出功能，不直接依赖 ExportLog。

1.3.1 持久化实体层（全部退化为纯数据实体）

User（用户类） — S 原则修正

属性	类型	说明
id	Long	用户ID（主键）
username	String	登录账号（唯一）
password	String	密码（加密存储）
realName	String	真实姓名
phone	String	手机号
status	Integer	状态：1=正常，0=禁用
roleId	Long	全局角色ID（外键→Role.id）
createdAt	DateTime	创建时间
updatedAt	DateTime	更新时间

方法：无（纯数据实体）

修正说明：原 login/logout/changePassword/hasPermission/getTasks 已全部迁移至对应 Service。

Role（角色类）

属性	类型	说明
id	Long	角色ID（主键）
roleCode	String	角色代码：SUPER_ADMIN / TASK_CREATOR / ANNOTATOR / ARBITRATOR
roleName	String	角色显示名称

方法：无

TaskAssignment（任务分配类） — S 原则修正

属性	类型	说明
id	Long	分配记录ID（主键）
taskId	Long	任务ID（外键→Task.id）
documentId	Long	文档ID（外键→TaskDocument.id）
userId	Long	用户ID（外键→User.id）
roleInTask	TaskRole	任务内身份（枚举）

属性	类型	说明
assignedBy	Long	分配者ID（外键→User.id）
assignedAt	DateTime	分配时间

方法：无（纯数据实体）

修正说明：原 `assign/revoke/getUserRoleInTask` 已迁移至 `TaskAssignmentService`。

TaskRole（任务角色枚举） — O 原则修正

1	ANNOTATOR	- 标注员
2	ARBITRATOR	- 裁定者
3	REVIEWER	- 审核员（预留扩展）

开闭原则：新增角色只需增加枚举值，无需新增接口方法。

GuideVersion（指南版本类）

属性	类型	说明
id	Long	版本ID（主键）
versionName	String	版本名称
description	String	版本说明
isActive	Boolean	是否为当前启用版本
createdBy	Long	创建人ID（外键→User.id）
createdAt	DateTime	创建时间

方法：无（纯数据实体，配置型数据）

Label（标签类） — L 原则修正

属性	类型	说明
id	Long	标签ID（主键）
guideVersionId	Long	所属指南版本ID
name	String	标签全称
abbr	String	标签简称
sortOrder	Integer	显示顺序
parentL1Id	Long	所属一级标签ID（二级标签特有）
level	Integer	标签层级：1=一级，2=二级

方法：无（纯数据实体）

工厂方法（保证 L 原则）：

工厂方法	说明
+ createL1(guideVersionId, name, abbr): Label	创建一级标签（parentL1Id 自动为 null）
+ createL2(guideVersionId, parentL1Id, name, abbr): Label	创建二级标签（parentL1Id 必填）

里氏替换修正：通过工厂方法确保 L1/L2 标签在创建时语义正确，调用方无需前置判断 level。

RelationType（关系类型类）

属性	类型	说明
id	Long	关系类型ID（主键）
guideVersionId	Long	所属指南版本ID
name	String	关系名称
abbr	String	关系简称
isBinary	Boolean	是否为二元关系
sortOrder	Integer	显示顺序

方法：无

Task（任务类） — S/O/D 原则修正

属性	类型	说明
id	Long	任务ID（主键）
title	String	任务名称
description	String	任务描述
status	TaskStatus	任务状态枚举
stage	TaskStage	当前阶段枚举
guideVersionId	Long	使用的指南版本ID（外键→GuideVersion.id）
creatorId	Long	创建者ID（外键→User.id）
createdAt	DateTime	创建时间
stageChangedAt	DateTime	阶段最近变更时间

方法（仅保留核心访问器）：

方法签名	说明
+ getId(): Long	获取任务ID
+ getTitle(): String	获取任务名称
+ getStatus(): TaskStatus	获取状态
+ getStage(): TaskStage	获取阶段

修正说明:

- 原 assignAnnotator/assignArbitrator → 迁移到 TaskAssignmentService.assignMember()
- 原 exportData() → 迁移到 ExportCoordinator.export(), Task 通过 IExportable 接口间接调用
- 原 saveLabelConfig/getLabelConfigHistory → 迁移到 LabelConfigService
- 原 advanceStage/canAdvance → 迁移到 TaskLifecycleService, 内部使用状态模式
- 原 labelConfigSnapshot JSON 字段 → 已移除, 由 LabelConfig 实体替代

TaskStatus (任务状态枚举)

1	PENDING	- 待分配
2	ANNOTATING	- 标注中
3	ARBITRATING	- 裁决中
4	COMPLETED	- 已完成
5	EXPORTABLE	- 可导出

TaskStage (任务阶段枚举)

1	CREATION	- 创建阶段
2	ANNOTATION	- 标注阶段
3	ARBITRATION	- 裁决阶段
4	OUTPUT	- 输出阶段

GlobalDocument (全局文档类)

属性	类型	说明
id	Long	全局文件ID (主键)
fileName	String	文件名
filePath	String	文件存储路径
fileType	String	文件类型: pdf/docx/txt
extractedText	String	提取的纯文本内容

属性	类型	说明
uploadedById	Long	上传者ID（外键→User.id）
uploadedAt	DateTime	上传时间

方法：无（纯数据实体）

TaskDocument（任务文档类）

属性	类型	说明
id	Long	任务文档ID（主键）
taskId	Long	所属任务ID（外键→Task.id）
sourceType	SourceType	数据来源：GLOBAL / UPLOAD
globalDocId	Long	引用全局文件ID（sourceType=GLOBAL时，外键→GlobalDocument.id）
fileName	String	原始文件名
filePath	String	上传文件存储路径
extractedText	String	最终用于标注的纯文本
extractedRange	String	自动提取范围描述
isManuallyCorrected	Boolean	是否手动修正过提取范围
uploadedAt	DateTime	添加时间

方法：无（纯数据实体）

修正说明：原 `createFromGlobal/createFromUpload/correctExtraction/getText` 已迁移至 `TaskDocumentService`。

LabelConfig（任务标签配置历史类） — O/D 原则修正

属性	类型	说明
id	Long	配置历史ID（主键）
taskId	Long	所属任务ID（外键→Task.id）
configJson	JSON	完整的标签配置快照
version	Integer	配置版本号
createdAt	DateTime	创建时间

方法：无（纯数据实体）

修正说明：已从 `Task.LabelConfigsnapshot` JSON 字段提取为独立实体，与 ER 图"任务标签配置历史表"精确映射。业务方法迁移至 `LabelConfigService`。

Proposition（命题类）

属性	类型	说明
id	Long	命题ID（主键）
documentId	Long	所属文档ID（外键→TaskDocument.id）
sequenceNo	Integer	在文档中的序号
startPos	Integer	起始字符位置
endPos	Integer	结束字符位置
selectedText	String	选中的文本片段
labelPath	String	标签路径
createdBy	Long	创建者ID（外键→User.id）
createdAt	DateTime	创建时间
updatedAt	DateTime	更新时间

方法：无（纯数据实体）

修正说明：原 `create/delete/updateLabel/renumber/getRelations` 已迁移至 `PropositionService`。

Relation（关系类）

属性	类型	说明
id	Long	关系ID（主键）
documentId	Long	所属文档ID（外键→TaskDocument.id）
relationType	String	关系类型（S/A/J/M/I）
isNested	Boolean	是否为嵌套关系
parentRelationId	Long	父关系ID（外键→Relation.id）
createdBy	Long	创建者ID（外键→User.id）
createdAt	DateTime	创建时间
updatedAt	DateTime	更新时间

方法：无（纯数据实体）

RelationMember（关系成员类）

属性	类型	说明
id	Long	成员明细ID（主键）
relationId	Long	所属关系ID（外键→Relation.id）
memberType	MemberType	成员类型：P=命题，R=关系
memberId	Long	成员ID
memberRole	MemberRole	角色：SOURCE / TARGET / MEMBER
memberOrder	Integer	参数顺序

方法：无（纯数据实体）

Annotation（标注结果快照类） — S/D 原则修正

属性	类型	说明
id	Long	快照ID（主键）
taskId	Long	所属任务ID（外键→Task.id）
documentId	Long	文档ID（外键→TaskDocument.id）
userId	Long	标注员ID（外键→User.id）
propositionData	JSON	命题数据快照（原始JSON字符串）
relationData	JSON	关系数据快照（原始JSON字符串）
submittedAt	DateTime	提交时间
isFinal	Boolean	是否为最终版本

方法：无（纯数据实体）

修正说明：

- 原 `submit/overwrite` → 迁移至 `AnnotationService`
- 原 `getPropositions/getRelations`（含JSON反序列化）→ 迁移至 `AnnotationDataConverter`
- 实体仅持有原始 JSON 字符串，不依赖任何序列化框架

Arbitration（裁决类）

属性	类型	说明
id	Long	裁决ID（主键）
taskId	Long	所属任务ID（外键→Task.id）

属性	类型	说明
documentId	Long	文档ID（外键→TaskDocument.id，唯一）
arbitratorId	Long	裁决者ID（外键→User.id）
finalPropositionData	JSON	最终命题数据快照
finalRelationData	JSON	最终关系数据快照
adoptedFrom	String	采纳来源
arbitratedAt	DateTime	裁决时间
isFinal	Boolean	是否为最终版本

方法：无（纯数据实体）

修正说明：原业务方法迁移至 `ArbitrationService`。

ExportLog（导出日志类）

属性	类型	说明
id	Long	导出ID（主键）
taskId	Long	任务ID（外键→Task.id）
exportType	String	导出格式
filePath	String	压缩包文件路径
fileSize	Long	压缩包大小
status	String	状态
progress	Integer	进度（0-100）
labelVersion	String	导出时标签版本快照
requestedBy	Long	请求导出用户ID（外键→User.id）
createdAt	DateTime	请求时间
completedAt	DateTime	完成时间

方法：无（纯数据实体）

ExportFile（导出文件明细类）

属性	类型	说明
id	Long	文件明细ID（主键）
exportLogId	Long	所属导出日志ID（外键→ExportLog.id）

属性	类型	说明
fileUrl	String	文件访问URL
expiryAt	DateTime	临时文件过期时间

方法：无（纯数据实体）

1.3.2 服务层（新增 — 单一职责 + 依赖倒转）

TaskLifecycleService（任务生命周期服务） — 实现 ITaskLifecycle

依赖	说明
TaskRepository	任务数据访问
TaskStageState	状态模式对象（组合）

方法签名	说明
+ advanceStage(taskId): void	推进阶段，委托给当前 State 对象
+ canAdvance(taskId): boolean	检查推进条件
+ getCurrentStage(taskId): TaskStage	获取当前阶段
+ addDocument(taskId, doc): void	向任务追加文档

单一职责：仅负责任务生命周期管理。

TaskAssignmentService（任务分配服务） — 实现 IAssignable — O 原则修正

依赖	说明
TaskAssignmentRepository	分配记录数据访问

方法签名	说明
+ assignMember(taskId, documentId, userId, roleType): TaskAssignment	统一分配入口
+ getAssignments(taskId): List<TaskAssignment>	获取分配列表
+ revokeAssignment(assignmentId): void	撤销分配
+ getUserRoleInTask(userId, taskId): TaskRole	查询用户角色

开闭原则修正：原 assignAnnotator() 和 assignArbitrator() 两个硬编码方法，合并为 assignMember(uid, docId, roleType) 统一入口。新增角色只需扩展 TaskRole 枚举。

LabelConfigService（标签配置服务）

依赖	说明
LabelConfigRepository	配置历史数据访问

方法签名	说明
+ saveConfig(taskId, configJson): LabelConfig	保存新配置版本
+ getHistoryByTask(taskId): List<LabelConfig>	获取配置变更历史
+ getVersion(taskId, version): LabelConfig	获取指定版本配置
+ getLatestByTask(taskId): LabelConfig	获取最新配置版本
+ rollbackTo(taskId, version): LabelConfig	回滚到指定版本

单一职责：从 Task 类中剥离，独立管理标签配置历史。

AnnotationService（标注服务） — S/D 原则修正

依赖	说明
AnnotationRepository	标注数据访问
AnnotationDataConverter	数据转换器（JSON ↔ 对象）

方法签名	说明
+ submit(taskId, documentId, userId, props, rels): Annotation	提交标注结果
+ overwrite(annotationId, props, rels): void	覆盖提交
+ getPropositions(annotationId): List<Proposition>	获取命题列表（委托 Converter）
+ getRelations(annotationId): List<Relation>	获取关系列表（委托 Converter）
+ getByDocumentAndUser(docId, userId): Annotation	按文档和用户查询

修正说明：submit/overwrite 从 Annotation 实体移出；getPropositions/getRelations 委托给 Converter，Annotation 实体不再依赖 JSON 序列化细节。

AnnotationDataConverter（标注数据转换器） — D 原则修正

方法签名	说明
+ deserializePropositions(json): List<Proposition>	JSON 反序列化为命题列表

方法签名	说明
+ deserializeRelations(json): List<Relation>	JSON 反序列化为关系列表
+ serializePropositions(props): String	命题列表序列化为 JSON
+ serializeRelations(rels): String	关系列表序列化为 JSON

依赖倒转修正：JSON 反序列化逻辑从 Annotation 实体中彻底移出。Annotation 实体只持有原始 JSON 字符串，不了解序列化格式。切换序列化方案（如 Protobuf）只需修改此转换器。

AuthService（认证服务） — 实现 IAuthService — S 原则修正

依赖	说明
UserRepository	用户数据访问
PasswordEncoder	密码加密器

方法签名	说明
+ login(username, password): AuthToken	用户登录验证
+ logout(token): void	退出登录
+ changePassword(userId, oldPwd, newPwd): boolean	修改密码

单一职责修正：从 User 实体中剥离认证逻辑。

PermissionService（权限服务） — 实现 IPermissionService — S 原则修正

依赖	说明
UserRepository	用户数据访问
RoleRepository	角色数据访问

方法签名	说明
+ hasPermission(userId, permissionCode): boolean	权限校验
+ checkRole(userId, roleCode): boolean	角色校验

单一职责修正：从 User 实体中剥离权限逻辑。

TaskQueryService（任务查询服务） — S 原则修正

依赖	说明
TaskRepository	任务数据访问

方法签名	说明
+ getTasksByUser(userId): List<Task>	获取用户参与的任务
+ getCreatedTasks(userId): List<Task>	获取用户创建的任务
+ getTaskDetail(taskId): TaskDetailDTO	获取任务详情

单一职责修正：替代原 `User.getTasks()`。

ArbitrationService（裁决服务）

依赖	说明
ArbitrationRepository	裁决数据访问
AnnotationRepository	标注数据访问

方法签名	说明
+ startArbitration(taskId, documentId, arbitratorId): Arbitration	开始裁决
+ adoptAll(arbitrationId, annotationId): void	全部采纳
+ adoptWithEdit(arbitrationId, props, rels): void	部分采纳并修改
+ confirmFinal(arbitrationId): void	确认最终裁定
+ compareAnnotations(ann1, ann2): List<Conflict>	比对差异

ExportCoordinator（导出协调器） — 实现 IExportService — O/D 原则修正

依赖	说明
AbstractExporter	模板方法模式对象（组合）
ExportLogRepository	导出日志数据访问

方法签名	说明
+ export(taskId, format, requestedBy): ExportResult	执行导出
+ getExportLogs(taskId): List<ExportLog>	获取导出历史

开闭原则修正：新增导出格式只需新增 `AbstractExporter` 子类，零侵入现有代码。

依赖倒转修正：Task 依赖 `IExportService` 接口，不直接依赖 `ExportLog` 具体类。
`ExportResult` 是封装了 `ExportLog` 的结果对象。

DocumentParserFactory（文档解析工厂） — O 原则修正

方法签名	说明
+ getParser(fileType): IDocumentParser	根据文件类型获取解析策略
+ registerParser(fileType, parser): void	注册新解析器

开闭原则修正：防止业务代码退化做 if-else 判断，确保始终通过接口调用。

1.3.3 状态模式类（新增 — 开闭原则）

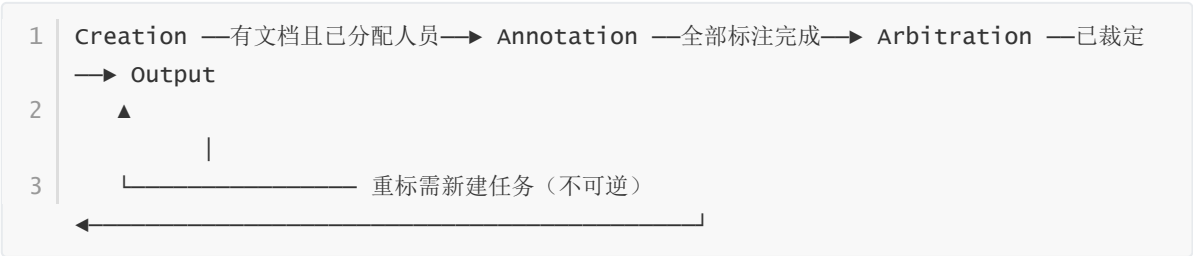
TaskStageState（状态接口）

方法签名	说明
+ canAssign(): boolean	是否允许分配
+ canAnnotate(): boolean	是否允许标注
+ canArbitrate(): boolean	是否允许裁决
+ canExport(): boolean	是否允许导出
+ advance(task): void	推进到下一阶段
+ getName(): String	获取阶段名称

CreationState → AnnotationState → ArbitrationState → OutputState

四个状态类的实现同初稿中的状态模式设计（已验证通过开闭原则），此处省略重复定义。

状态转换规则：



1.3.4 模板方法模式类（新增 — 开闭原则）

AbstractExporter（抽象导出器）

实现同初稿中的模板方法模式设计（已验证通过开闭原则），此处省略重复定义。

关键设计：export() 方法标记为 final，子类不可覆盖，确保流程骨架固定。

1.3.5 观察者模式类（新增 — 解耦）

AnnotationWorkspace（标注工作台 — 实现 IAnnotationSubject）

实现同初稿中的观察者模式设计，此处省略重复定义。

GraphView / PropositionListView（实现 IAnnotationObserver）

运行时视图对象，无 ER 实体对应，同初稿设计。

1.3.6 论证图示模块（运行时对象，无 ER 对应）

ArgumentationGraph / GraphNode / GraphEdge

同初稿设计，此处省略。

1.4 类之间的关系汇总

1.4.1 持久化实体间关系（与 ER 图一致）

关系类型	源类	目标类	基数	对应 ER 图关系
关联	User	Role	N:1	用户表—拥有一全局角色表
关联	User	Task	1:N	用户表—创建—任务表
关联	Task	GuideVersion	N:1	任务表—被引用—指南版本表
关联	Task	TaskDocument	1:N	任务表—包含—任务文档表
关联	TaskDocument	GlobalDocument	N:1	任务文档表—被引用—全局文件库
关联	Task	TaskAssignment	1:N	任务表—分配—任务分配表
关联	TaskAssignment	User	N:1	任务分配表—被分配—用户表
关联	GuideVersion	Label	1:N	指南版本表—定义—一级/二级标签表
关联	Label(L1)	Label(L2)	1:N	一级标签表—包含—二级标签表
关联	GuideVersion	RelationType	1:N	指南版本表—定义—关系类型表
关联	TaskDocument	Proposition	1:N	任务文档表—包含—命题表
关联	TaskDocument	Relation	1:N	任务文档表—包含—关系表

关系类型	源类	目标类	基数	对应 ER 图关系
关联	Relation	RelationMember	1:N	关系表—包含成员—关系成员明细表
关联	Proposition	RelationMember	1:N (间接)	命题表—作为成员—关系成员明细表
关联	Relation	Relation	1:N	关系表—自嵌套—关系表
关联	TaskDocument	Annotation	1:N	任务文档表—产生—标注结果快照表
关联	User	Annotation	1:N	用户表—提交—标注结果快照表
关联	TaskDocument	Arbitration	1:1	任务文档表—拥有一裁决结果表
关联	Task	ExportLog	1:N	任务表—发起导出—导出日志表
关联	ExportLog	ExportFile	1:N	导出日志表—包含—导出文件明细表
关联	Task	LabelConfig	1:N	任务表—拥有一任务标签配置历史表

1.4.2 服务层与实体层关系

关系类型	源类	目标类	说明
依赖	TaskLifecycleService	Task	服务依赖实体数据
依赖	TaskLifecycleService	TaskStageState	组合，状态模式
依赖	TaskAssignmentService	TaskAssignment	服务操作实体
依赖	LabelConfigService	LabelConfig	服务操作实体
依赖	AnnotationService	Annotation	服务操作实体
依赖	AnnotationService	AnnotationDataConverter	组合，数据转换
依赖	ExportCoordinator	AbstractExporter	组合，模板方法模式
依赖	ExportCoordinator	ExportLog	记录导出结果
依赖	AuthService	User	认证服务依赖用户实体
依赖	PermissionService	User / Role	权限服务依赖用户/角色
依赖	TaskQueryService	Task	查询服务依赖任务实体

1.4.3 接口实现关系

关系类型	实现类	接口	说明
实现	TaskLifecycleService	ITaskLifecycle	生命周期管理
实现	TaskAssignmentService	IAssignable	任务分配
实现	ExportCoordinator	IExportService	导出服务
实现	AuthService	IAuthService	认证
实现	PermissionService	IPermissionService	权限
实现	AnnotationWorkspace	IAnnotationSubject	观察者模式—主题
实现	GraphView / PropListView	IAnnotationObserver	观察者模式—观察者
实现	PdfParser	IDocumentParser	策略模式
实现	WordParser	IDocumentParser	策略模式
实现	TextParser	IDocumentParser	策略模式
实现	PdfParser	IReasonExtractable	接口隔离
实现	WordParser	IReasonExtractable	接口隔离
实现	CreationState / AnnotationState / ArbitrationState / OutputState	TaskStageState	状态模式
实现	ExcelExporter / JsonExporter / ImageExporter	AbstractExporter	模板方法模式

1.5 SOLID 整改对照表

原则	原违规项	修正措施	修正位置
S-1	Task 5大职责聚合	拆出 TaskAssignmentService、LabelConfigService、ExportCoordinator、TaskLifecycleService	3.2 服务层
S-2	Annotation 混入业务+序列化	拆出 AnnotationService、AnnotationDataConverter	3.2 服务层
S-3	User 混入认证+权限+查询	拆出 AuthService、PermissionService、TaskQueryService	3.2 服务层
O-1	新增导出格式改现有代码	应用模板方法模式：AbstractExporter + 子类	3.4 模板方法模式

原则	原违规项	修正措施	修正位置
O-2	新增角色需加硬编码方法	assignAnnotator/assignArbitrator → assignMember(uid, docId, roleType)	IAssignable 接口
O-3	labelConfigSnapshot JSON死字段	提取 LabelConfig 独立实体	3.1 LabelConfig
O-4	Parser退化风险	增加 DocumentParserFactory 工厂	3.2 DocumentParserFactory
L-1	Label类语义双重性	增加工厂方法 createL1/createL2 保证创建语义	Label 工厂方法
I-1	Task方法横跨4领域	拆分为 ITaskLifecycle/IAssignable/IExportable 等细粒度接口	3.0 接口层
I-2	Parser未来变胖风险	拆分为 IDocumentParser / IReasonExtractable	3.0 接口层
D-1	Task直接依赖 ExportLog	Task → IExportService → ExportCoordinator → ExportLog	3.0/3.2 接口+服务层
D-2	Annotation依赖JSON序列化	JSON反序列化移至 AnnotationDataConverter	3.2 AnnotationDataConverter
D-3	Task依赖JSON存储格式	labelConfigSnapshot 移除, 改为 LabelConfig 实体	3.1 LabelConfig

1.6 SOLID检查清单

1.6.1 S — 单一职责原则 (Single Responsibility Principle)

检查问题：有没有类承担了过多职责？

结论：违反，共发现 3 处。

类名	涉及职责数	具体职责列表	违反说明	修正方案
Task	5	① 任务生命周期管理（创建、阶段推进） ② 人员分配（assignAnnotator、assignArbitrator） ③ 标签配置管理（labelConfigSnapshot 字段） ④ 数据导出（exportData） ⑤ 多维度状态判断（canAdvance、isLabelSystemLocked）	一个类同时负责"任务自身数据"、"人员分配流程"、"配置版本控制"、"导出协调"四大不同领域的逻辑，任何一个领域的变更都可能导致 Task 类被修改，引起不必要的回归风险。	拆分为 5 个专门类/服务： ① Task 保留核心属性 + 生命周期方法 ② 新增 TaskAssignmentService 处理分配逻辑 ③ 新增 LabelConfigService + LabelConfig 实体处理配置历史 ④ 新增 ExportCoordinator 委托导出流程 ⑤ 状态判断逻辑下沉到状态模式各子类
Annotation	3	① 数据实体（存储命题/关系快照） ② 业务行为（submit、overwrite） ③ 序列化/反序列化（getPropositions、getRelations）	数据实体类混入业务逻辑和数据转换逻辑，违反了分层设计原则。序列化方式变更（如从 JSON 改为 Protocol Buffers）会迫使数据实体类修改。	拆分为两层： ① Annotation 仅保留数据属性（POJO/Entity） ② 新增 AnnotationService 处理 submit/overwrite 业务逻辑 ③ 新增 AnnotationDataConverter 负责 JSON ↔ 对象转换
User	3	① 用户数据载体 ② 认证逻辑（login、logout） ③ 权限判断（hasPermission） ④ 任务查询（getTasks）	领域模型类（User）混入横切关注点（认证、权限）和业务查询逻辑，导致领域层与基础设施层耦合。	拆分： ① User 仅保留数据属性 ② 新增 AuthService 处理 login/logout ③ 新增 PermissionService 处理 hasPermission ④ getTasks 移至 TaskQueryService

1.6.2 O — 开闭原则（Open/Closed Principle）

检查问题：新增需求类型是否需要修改现有代码？

结论：违反，共发现 4 处。

场景	是否违反	违反说明	修正方案
新增导出格式 (如增加 XML 导出)	是	原 <code>Task.exportData(format)</code> 方法需要直接修改 <code>Task</code> 类来支持新格式，违反"对扩展开放"。	使用 模板方法模式 ： ① 定义抽象类 <code>AbstractExporter</code> ② 各格式 (Excel/JSON/PNG/SVG) 继承并实现 <code>transform()</code> 和 <code>generateFile()</code> ③ <code>Task</code> 仅持有 <code>Exporter</code> 接口引用，新增格式零侵入
新增任务角色 (如增加"审核员"角色)	是	原 <code>Task</code> 类中 <code>assignAnnotator()</code> 和 <code>assignArbitrator()</code> 是两个独立硬编码方法，新增角色需再增加一个方法。	使用统一入口： ① 改为 <code>assignMember(userId, documentId, roleType: Enum)</code> 单方法 ② 角色类型通过 <code>TaskRole</code> 枚举扩展 ③ 或进一步使用 策略模式 ，不同角色对应不同 <code>AssignmentStrategy</code>
标签配置历史追溯	是	原 <code>Task</code> 类用 <code>labelConfigSnapshot: JSON</code> 字段存储配置，是一个"死字段"。新增"查看配置变更历史"功能时，必须将 JSON 字段重构为独立实体，修改 <code>Task</code> 类结构。	提取独立实体 <code>LabelConfig</code> (已修正)： ① <code>LabelConfig</code> 与 <code>Task</code> 为 1:N 关联 ② 通过 <code>LabelConfigService</code> 管理版本历史 ③ <code>Task</code> 类不再直接管理配置数据
新增文件解析格式 (如增加 HTML 导入)	轻度	原类图中虽然定义了 <code>DocumentParser</code> 接口和 <code>PdfParser</code> / <code>WordParser</code> / <code>TextParser</code> 三个实现，但接口设计尚未完全落实，存在退化风险——如果开发者在 <code>TaskDocument</code> 中直接根据文件类型做 if-else 判断选择解析器，则开闭原则被破坏。	确保 <code>DocumentParser</code> 接口正确使用： ① <code>TaskDocument</code> 持有 <code>DocumentParser</code> 接口引用而非具体类 ② 通过 <code>DocumentParserFactory</code> 在运行时根据文件类型动态选择策略 ③ 新增 HTML 格式只需新增 <code>HtmlParser implements DocumentParser</code>

1.6.3 L — 里氏替换原则 (Liskov Substitution Principle)

检查问题：子类是否可以替换父类使用？

结论：轻度违反，共发现 1 处。

类/继承关系	是否违反	违反说明	修正方案
<code>Label</code> 类同时用于一级标签和二级标签	轻度违反	原设计用一个 <code>Label</code> 类表达两种概念：一级标签 (SF/GF/SM/GM) 和二级标签 (GM-L/GM-I 等)。通过 <code>level</code> 字段和 <code>parentL1Id</code> 字段区分。这导致： ① 一级标签的 <code>parentL1Id</code> 字段永远为空/无意义，存在语义污染 ② 调用方必须在使用前判断 <code>isLevel2()</code>，无法无差别替换使用 ③ ER 图中明确区分"一级标签表"和"二级标签表 (仅GM)"，类图应尊重这一语义边界	方案 A (推荐)：保留统一 <code>Label</code> 类，但通过工厂方法保证创建时语义正确： ① <code>Label.createL1(...)</code> 创建时不传 <code>parentL1Id</code> ② <code>Label.createL2(...)</code> 创建时必须填 <code>parentL1Id</code> ③ 文档层面明确说明 <code>parentL1Id</code> 对 L1 标签无意义 方案 B (更严格)：拆分为 <code>L1Label extends Label</code> 和 <code>L2Label extends Label</code>，通过继承表达层级差异

注：原类图中没有大量继承层次结构，所以里氏替换的违规点相对较少。上述问题属于"语义层面"的替换障碍，而非典型的继承 override 破坏前置/后置条件。

1.6.4 I — 接口隔离原则 (Interface Segregation Principle)

检查问题：有没有接口太"胖"，包含了不需要的方法？

结论：违反，共发现 2 处。

接口/类	是否违反	违反说明	修正方案
隐含的 Task 接口 (如果将 Task 抽象为接口)	是	原 Task 类的方法集包含了生命周期、分配、配置、导出四大不相关领域的方法。如果将其行为抽象为接口，下游依赖方会被迫依赖大量不需要的的方法： ① "任务列表查询"只需要 getStatus()、getTitle() 等读取方法 ② "人员分配"只需要 assignMember() 相关方法 ③ "导出功能"只需要 exportData() 相关方法	将 Task 的行为拆分为多个细粒度接口： ① TaskReadable — 读取任务基本信息 ② TaskLifecycle — 阶段推进相关 ③ Assignable — 人员分配相关 ④ Configurable — 标签配置相关 ⑤ Exportable — 数据导出相关 不同依赖方只依赖所需接口
DocumentParser 接口 (潜在风险)	潜在风险	原接口只有 parse() 和 extractReasonSection() 两个方法，当前尚不算"胖接口"。但如果未来增加更多方法 (如 validateFormat()、extractMetadata()、getPageCount())，则对于纯文本解析器来说，getPageCount() 等方法将是无意义的。	预防性拆分： ① DocumentParser — 只保留 parse() ② ReasonExtractable — 保留 extractReasonSection() ③ PageCountable — 新增 getPageCount() 各解析器按需实现多个接口

1.6.5 D — 依赖倒转原则 (Dependency Inversion Principle)

检查问题：高层模块是否直接依赖了低层模块的具体实现？

结论：违反，共发现 3 处。

高层模块	直接依赖的低层模块	是否违反	违反说明	修正方案
Task (领域层)	ExportLog (具体类)	是	Task.exportData(): ExportLog 直接返回 ExportLog 具体类，而非抽象。当导出逻辑变更 (如异步导出、分片导出) 时，领域层 Task 被迫修改。	引入接口：Task 依赖 ExportService 接口： interface ExportService { export(taskId, format): ExportResult; } ExportLog 降为 ExportResult 的一种实现
Annotation (实体层)	JSON 序列化机制	是	Annotation.getPropositions() / getRelations() 内部直接进行 JSON 反序列化，实体层直接依赖了数据格式细节。JSON 格式变更或切换序列化方案时，实体类必须修改。	依赖倒置：实体层只持有 String / byte[] 原始数据，反序列化由 AnnotationDataConverter 服务完成，实体通过 Converter 接口间接获取转换结果

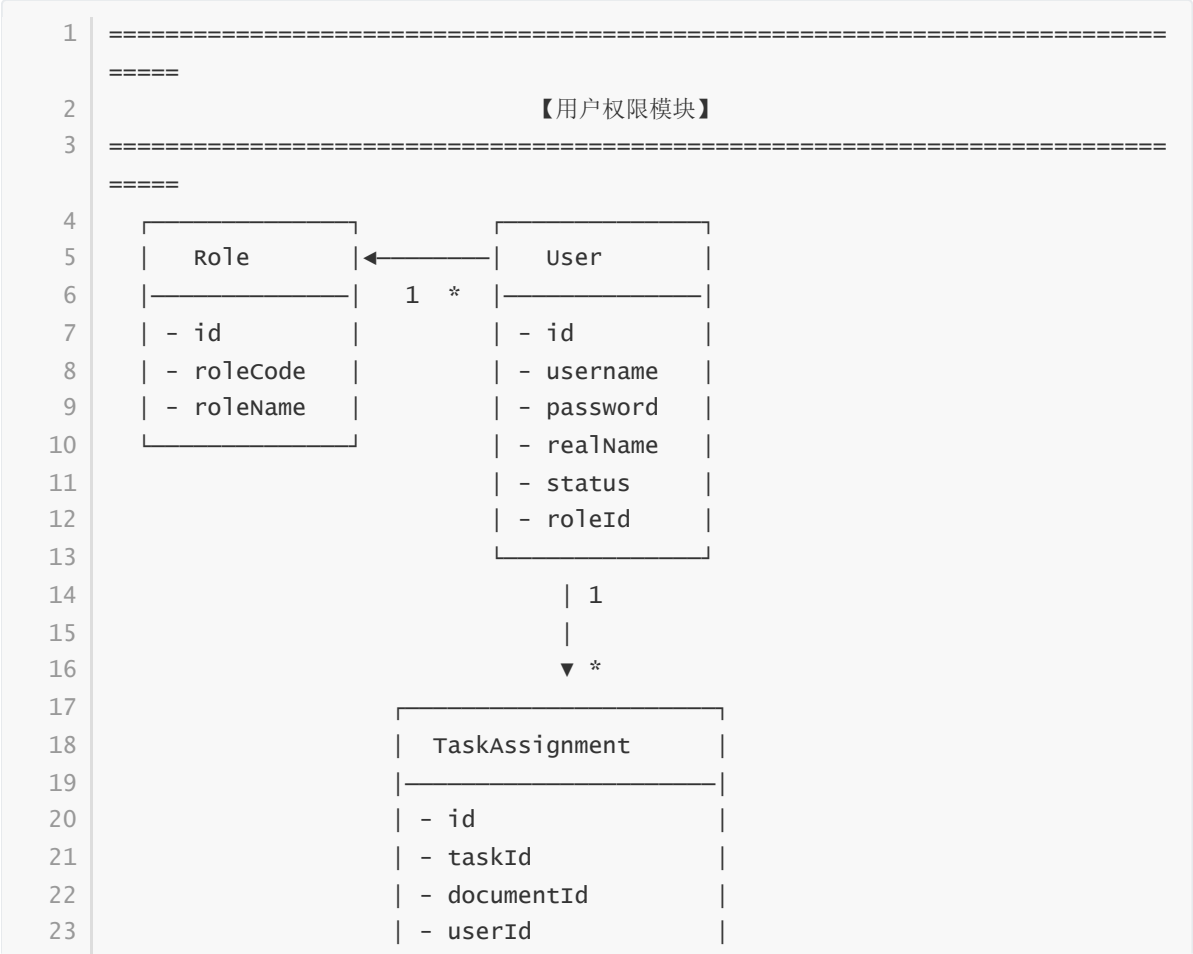
高层模块	直接依赖的低层模块	是否违反	违反说明	修正方案
Task（领域层）	LabelConfigSnapshot: JSON	是	领域对象 Task 直接依赖了 JSON 数据格式的具体存储方式，配置存储方式变更（如改为独立表存储）时，Task 类结构被迫修改。	提取 LabelConfig 独立实体，Task 通过 LabelConfigRepository 接口获取配置数据，Task 本身不再持有配置存储细节

1.6.6 SOLID 违规汇总表

SOLID 原则	检查问题	AI 设计是否违反	违反说明	修正方案
S — 单一职责	Task类是否承担了过多职责？	违反	Task类同时负责任务生命周期、人员分配、标签配置管理、数据导出、状态判断5大类职责，任何一类变更都可能修改Task类	拆分为 Task（核心数据）+ TaskAssignmentService + LabelConfigService + ExportCoordinator
S — 单一职责	Annotation类是否承担了过多职责？	违反	数据实体类混入业务逻辑（submit/overwrite）和数据转换逻辑（JSON序列化/反序列化）	拆分为 Annotation（纯数据实体）+ AnnotationService（业务逻辑）+ AnnotationDataConverter（数据转换）
S — 单一职责	User类是否承担了过多职责？	违反	领域模型类混入认证（login/logout）、权限（hasPermission）、业务查询（getTasks）三类横切/业务逻辑	拆分为 User（纯数据）+ AuthService + PermissionService + TaskQueryService
O — 开闭原则	新增导出格式是否需要修改现有代码？	违反	Task.exportData(format) 需要在Task类中增加对新格式的处理逻辑	使用模板方法模式：AbstractExporter 定义骨架，各格式继承实现
O — 开闭原则	新增任务角色是否需要修改现有代码？	违反	assignAnnotator() 和 assignArbitrator() 是两个硬编码方法，新增角色需增加新方法	统一为 assignMember(userId, docId, roleType: Enum) 或策略模式
O — 开闭原则	新增配置历史追溯功能是否需要修改现有代码？	违反	labelConfigsnapshot 是JSON死字段，增加历史版本功能需重构Task类结构	提取独立的 LabelConfig 实体类，1:N关联Task
O — 开闭原则	新增文件解析格式是否需要修改现有代码？	轻度风险	虽然定义了DocumentParser接口，但存在退化风险（可能在TaskDocument中做if-else）	确保通过 DocumentParserFactory 动态选择策略，不侵入业务代码
L — 里氏替换	Label类用于两种标签层级是否有替换问题？	轻度违反	一级标签的 parentL1Id 字段无意义，调用方必须先判断 isLevel2() 才能安全使用	方案A：通过工厂方法（createL1/createL2）确保创建语义正确；方案B：拆分为 L1Label/L2Label继承Label

SOLID 原则	检查问题	AI 设计是否违反	违反说明	修正方案
I — 接口隔离	是否存在太"胖"的接口？	违反	Task类的方法集横跨生命周期/分配/配置/导出四个领域，若抽象为接口则依赖方被迫依赖不需要的方法	拆分为 TaskReadable / TaskLifecycle / Assignable / Configurable / Exportable 等细粒度接口
I — 接口隔离	DocumentParser 接口未来是否可能变胖？	潜在风险	当前尚合理，但未来增加 validateFormat/getPageCount 等方法后，将成为胖接口	预防性拆分为 DocumentParser / ReasonExtractable / PageCountable 等独立接口
D — 依赖倒转	Task领域层是否直接依赖低层具体实现？	违反	<code>Task.exportData()</code> : <code>ExportLog</code> 直接依赖 <code>ExportLog</code> 具体类，而非导出服务抽象	引入 <code>ExportService</code> 接口，Task依赖接口而非具体实现
D — 依赖倒转	Annotation实体层是否直接依赖序列化细节？	违反	JSON反序列化逻辑硬编码在实体类中，实体直接依赖数据格式	将反序列化逻辑移至 <code>AnnotationDataConverter</code> ，实体通过接口间接获取
D — 依赖倒转	Task是否直接依赖JSON存储方式？	违反	<code>labelConfigSnapshot</code> : <code>JSON</code> 使领域对象直接依赖了具体存储格式	提取 <code>LabelConfig</code> 独立实体，Task通过 <code>Repository</code> 接口获取配置

1.6.7 原始生成



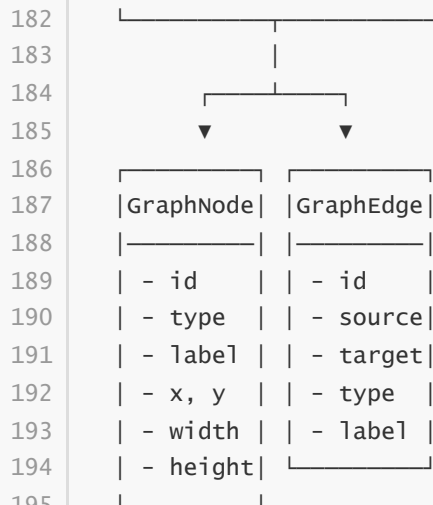


```
120 |-----|
121 | - id |
122 | - taskId |
123 | - documentId |
124 | - userId |
125 | - propositionData | (JSON快照)
126 | - relationData | (JSON快照)
127 | - submittedAt |
128 | - isFinal |
129 |-----|
130
131 =====
132
133 =====
134
135 |-----|
136 | Arbitration |
137 |-----|
138 | - id |
139 | - taskId |
140 | - documentId |
141 | - arbitratorId |
142 | - finalPropositionData |
143 | - finalRelationData |
144 | - adoptedFrom | (标注员ID / "MANUAL")
145 | - arbitratedAt |
146 | - isFinal |
147 |-----|
148
149 =====
150
151 =====
152
153 |-----| |-----|
154 | ExportLog | | ExportFile |
155 |-----| |-----|
156 | - id | | - id |
157 | - taskId | | - exportLogId |
158 | - exportType | | - fileUrl |
159 | - filePath | | - expiryAt |
160 | - fileSize | |-----|
161 | - status |
162 | - progress |
163 | - labelVersion |
164 | - requestedBy |
165 |-----|
166
167 =====
168
169 =====
170
171 |-----|
172 | ArgumentationGraph |
173 |-----|
```

```

170 |-----|
171 | - documentId |
172 | - nodes[] |
173 | - edges[] |
174 |-----|
175 | + addNode() |
176 | + addEdge() |
177 | + removeNode() |
178 | + removeEdge() |
179 | + layout() |
180 | + exportToImage() |
181 | + exportToSVG() |

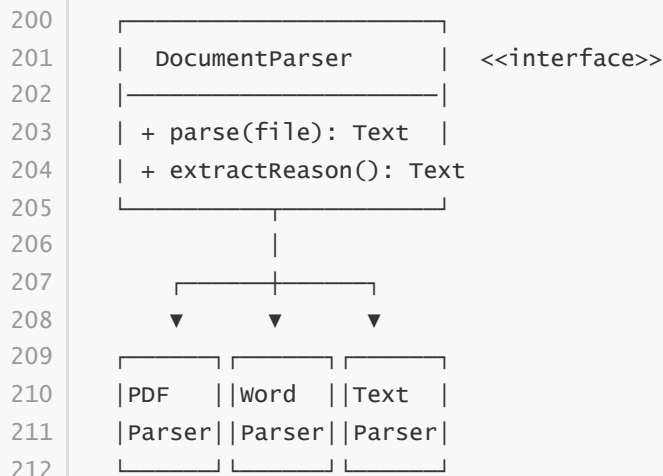
```



```

196 =====
197 =====
198                                     【文本处理模块】
199 =====
200 =====

```



1.6.8 修改方案

#	原则	违规点	修正措施
1	S	Task 5大职责聚合	拆出 TaskLifecycleService、TaskAssignmentService、LabelConfigService、ExportCoordinator
2	S	Annotation 混入业务+序列化	拆出 AnnotationService、AnnotationDataConverter
3	S	User 混入认证+权限+查询	拆出 AuthService、PermissionService、TaskQueryService
4	O	新增导出格式改现有代码	模板方法模式：AbstractExporter + 子类
5	O	新增角色需加硬编码方法	assignMember(uid, docId, roleType) 统一入口
6	O	labelConfigSnapshot JSON死字段	提取 LabelConfig 独立实体
7	O	Parser退化风险	DocumentParserFactory 强制走接口
8	L	Label类语义双重性	工厂方法 createL1() / createL2() 保证创建语义
9	I	Task方法横跨4领域	拆分为 ITaskLifecycle / IAssignable / IExportable 等
10	I	Parser未来变胖风险	拆分为 IDocumentParser / IReasonExtractable
11	D	Task直接依赖 ExportLog	Task → IExportService → ExportCoordinator → ExportLog
12	D	Annotation依赖JSON序列化	JSON转换移至 AnnotationDataConverter，实体只存原始字符串
13	D	Task依赖JSON存储格式	LabelConfigSnapshot 移除，改为 LabelConfig 实体

1.7 设计模式应用

1.7.1 状态模式 (State Pattern)

1.7.1.1 应用场景：任务四阶段生命周期管理

系统中每个任务具有严格的四阶段生命周期：

1

[创建阶段 CREATION] → [标注阶段 ANNOTATION] → [裁决阶段 ARBITRATION] → [输出阶段 OUTPUT]

核心约束：

- 四阶段流程**不可逆**，进入下一阶段后不可回退
- 每个阶段有各自的业务规则和校验条件
- 阶段推进需要满足特定前提（如所有标注员完成标注，才能进入裁决阶段）
- 不同阶段的 UI 操作按钮状态不同（如待分配状态显示"分配"，标注中显示"查看"）

`Task` 类本身退化为纯数据实体，生命周期行为由 `TaskLifecycleService` 负责。
`TaskLifecycleService` 内部持有 `TaskStageState` 接口引用，根据当前阶段动态指向 `CreationState`、`AnnotationState`、`ArbitrationState` 或 `OutputState` 具体实现。

1.7.1.2 为什么在这里使用状态模式

设计理由	说明
阶段行为差异大	每个阶段允许的操作完全不同：创建阶段可修改标签配置、分配人员；标注阶段仅允许标注操作；裁决阶段仅允许裁定操作；输出阶段仅允许导出。状态模式将每个阶段的行为封装在独立的状态类中。
状态转换有严格规则	不是任意状态都能互转。 <code>advance()</code> 方法在每个状态类中独立实现校验逻辑（如 <code>AnnotationState.advance()</code> 需检查所有标注员是否完成），非法转换在当前状态内直接拒绝。
避免大量条件判断	不用在每个 <code>Service</code> 方法里写 <code>if (stage == "ANNOTATION")</code> 的条件分支。 <code>canAnnotate()</code> 、 <code>canArbitrate()</code> 等方法直接委托给当前 <code>State</code> 对象回答。
符合开闭原则	未来若增加新的阶段（如"复核阶段"），只需新增 <code>TaskStageState</code> 实现类，不修改 <code>TaskLifecycleService</code> 和现有状态类。
与服务层架构配合	<code>Task</code> 纯数据实体不持有行为， <code>TaskLifecycleService</code> 通过组合 <code>State</code> 对象实现生命周期管理，职责边界清晰。

1.7.1.3 如果不用这个模式会怎样

问题	具体影响
满屏 if-else	<code>canAssign()</code> 、 <code>canAnnotate()</code> 等方法中写满 <code>if (stage === "CREATION") ... else if ...</code> ，代码可读性极差，且条件分支数量 = 阶段数 × 受阶段影响的操作数 ≈ 4×5 = 20+ 个分支
阶段规则分散	阶段转换逻辑散落在 <code>TaskLifecycleService</code> 的各方法中，修改一处校验规则容易遗漏其他关联处
违反开闭原则	新增阶段时需要修改所有包含阶段判断的代码，引入回归风险。每加一个阶段，需修改 5-10 处条件分支
状态转换不可控	容易出现非法状态跳转（如从创建直接跳到输出），且难以在编译期发现，只能在运行时通过测试捕获
单元测试困难	需要覆盖所有阶段的排列组合，测试用例数量随阶段数呈指数增长。4 个阶段需覆盖 16+ 种转换场景

问题	具体影响
与纯数据实体架构冲突	若不用状态模式，生命周期行为可能被迫塞回 Task 实体中，导致 Task 再次违反单一职责原则

1.7.2 观察者模式（Observer Pattern）

1.7.2.1 应用场景：标注数据变更与论证图示自动联动

系统核心交互需求：

- 当标注员**新增/删除命题**时，右侧论证图示区自动新增/删除对应节点
- 当标注员**新增/删除关系**时，图示区自动新增/删除对应边
- 当标注员**修改标签**时，图示区节点显示同步更新
- 图示区仅允许**拖拽节点位置**，不可独立新建/修改关系（双向同步）
- 同时，命题列表视图、关系列表视图也需要同步更新

即：**命题列表、关系列表、论证图示三者必须保持实时同步。**

`Annotationworkspace` 实现 `IAnnotationSubject` 接口，作为被观察的主题。`GraphView`、`PropositionListView`、`RelationListView` 实现 `IAnnotationObserver` 接口，作为观察者。标注操作触发事件后，`Annotationworkspace` 统一通知所有已注册的观察者。

1.7.2.2 为什么在这里使用观察者模式

设计理由	说明
一对多依赖关系	一个标注数据变更事件（如"命题3被删除"）需要同时通知命题列表、关系列表、图示区三个视图。观察者模式天然支持一个主题对应多个观察者的结构。
解耦数据与视图	标注核心模块（ <code>Annotationworkspace</code> ）不需要知道图示模块（ <code>GraphView</code> ）的存在。两者通过 <code>IAnnotationSubject</code> / <code>IAnnotationObserver</code> 接口交互。图示模块可以独立开发、测试、替换。
实时响应需求	系统需求明确要求标注操作后"实时自动生成图示"。观察者模式的事件驱动机制天然满足实时性要求。
支持视图扩展	未来可能新增"大纲视图""统计面板"等，只需实现 <code>IAnnotationObserver</code> 接口并注册到 <code>Annotationworkspace</code> ，无需修改标注模块的任何代码，符合开闭原则。
与服务层架构配合	观察者模式限定在运行时视图层，不影响持久化实体设计。 <code>AnnotationService</code> 负责业务逻辑， <code>Annotationworkspace</code> 负责视图联动，职责分离清晰。

1.7.2.3 如果不用这个模式会怎样

问题	具体影响
强耦合	<code>AnnotationWorkspace</code> 必须直接调用 <code>GraphView.addNode()</code> 、 <code>GraphView.removeEdge()</code> 等具体方法，两者紧密耦合。修改图示模块的接口签名会导致标注模块也要修改。
难以维护	每新增一个需要同步的视图（如"大纲视图"），都要修改 <code>AnnotationWorkspace</code> 的代码去调用新视图的同步方法，违反开闭原则。
代码重复	每个标注操作（增删改）都要在多个地方手动调用同步方法。例如删除命题时，需手动调用：命题列表更新、关系列表更新（级联删除）、图示区更新、序号重排——共 4 处。漏掉任何一处都会导致界面数据不一致。
性能隐患	无法灵活控制通知时机。例如批量删除 10 个命题时，不用观察者模式会触发 10 次重绘；使用观察者模式可在 <code>AnnotationWorkspace</code> 层做批量事件合并或防抖处理。
测试困难	测试标注功能时必须同时搭建图示环境，无法独立对 <code>AnnotationService</code> 做单元测试。测试 <code>GraphView</code> 时也必须构造完整的标注数据流。

具体场景举例：若不用观察者模式，当用户删除命题 3 时，需要在删除逻辑中**依次手动调用**：命题列表更新（移除项+序号重排）、关系列表更新（级联删除涉及命题3的关系）、图示区更新（移除节点+重排）、序号同步——共 4 处同步代码。漏掉任何一处都会导致界面与数据不一致，且这种 bug 极难排查。

1.7.3 策略模式 (Strategy Pattern)

1.7.3.1 应用场景：多格式文档解析

系统需要支持三种文件格式的导入和文本提取：

文件格式	解析方式	特点
PDF	pdf-parse 等库	需处理分页、表格、页眉页脚
Word (docx)	mammoth/docx4j 等库	需处理样式、批注、修订标记
纯文本 (txt)	直接读取	无需特殊处理

核心需求：

- 根据文件类型自动选择对应的解析策略
- 统一提取"裁判理由 / 本院认为"部分
- 支持后续扩展更多格式（如 HTML、Markdown）

- 解析逻辑与任务管理模块解耦

`DocumentParserFactory` 负责根据文件类型在运行时选择对应的 `IDocumentParser` 实现。
`TaskDocumentService` 依赖 `IDocumentParser` 接口而非具体实现类。接口已按接口隔离原则拆分为 `IDocumentParser` (仅含 `parse()`) 和 `IReasonExtractable` (含 `extractReasonSection()`) , 各解析器按需实现一个或两个接口。

1.7.3.2 为什么在这里使用策略模式

设计理由	说明
消除条件分支	不用在解析方法里写大量 <code>if (type === "pdf") ... else if (type === "docx") ...</code> 。 <code>DocumentParserFactory</code> 封装选择逻辑, <code>TaskDocumentService</code> 始终通过统一接口调用。
算法独立封装	每种格式的解析算法封装在独立类中 (<code>PdfParser</code> 、 <code>WordParser</code> 、 <code>TextParser</code>) , 互不干扰。PDF 的分页处理、Word 的样式清洗等专属逻辑不会污染通用代码。
易于扩展	新增文件格式只需新增一个实现 <code>IDocumentParser</code> 的类并在 <code>DocumentParserFactory</code> 中注册, 符合开闭原则。现有代码零侵入。
运行时动态选择	根据上传文件的类型在运行时动态选择解析策略, 用户无感知。
便于单元测试	每种解析策略可独立测试, Mock 数据更容易构造。 <code>TaskDocumentService</code> 测试时可用 <code>MockParser</code> 替代真实解析器。
与接口隔离原则配合	<code>IDocumentParser</code> 仅保留 <code>parse()</code> , <code>IReasonExtractable</code> 独立声明。纯文本解析器无需实现 <code>extractReasonSection()</code> (其逻辑与 PDF/Word 不同) , 避免被迫实现无意义的方法。

1.7.3.3 如果不用这个模式会怎样

问题	具体影响
巨大的条件分支	<code>parse()</code> 方法内需要写大量 <code>if/else</code> 判断文件类型。当前 3 种格式就需要 3 个分支, 每种格式 2-3 个方法 (<code>parse</code> + <code>extractReason</code> + 可能的辅助方法) , 方法体越来越长。
违反开闭原则	新增格式时必须修改 <code>TaskDocumentService</code> 类 (或其他承载解析逻辑的类) , 增加回归测试成本。每次新增格式都要重新测试已有的解析流程。
代码复用困难	裁判理由提取逻辑与文件类型判断纠缠在一起。PDF 和 Word 的提取逻辑可能有 80% 相似, 但由于嵌在各自的 <code>if</code> 分支中, 无法提取公共方法复用。
测试复杂	测试 PDF 解析时必须同时加载 Word 解析的代码路径, 无法隔离测试单一格式。一个格式的 bug 可能影响其他格式的测试。
格式专属逻辑污染通用代码	PDF 的分页处理、Word 的样式清洗等逻辑散落在通用流程中, 降低可读性。新开发者需要阅读整个 <code>if-else</code> 链才能理解某个格式的完整处理流程。

问题	具体影响
退化风险	即使最初设计时有接口意识，但在迭代压力下，开发者很容易在 <code>TaskDocumentService</code> 中直接写 <code>if (fileType === "pdf")</code> 的临时代码，导致设计逐渐退化为硬编码。 <code>DocumentParserFactory</code> 的存在从架构层面阻止了这种退化。

具体场景举例：当前支持 3 种格式，若未来扩展到 6 种格式（增加 HTML、Markdown、EPUB），不用策略模式时 `parse()` 方法的条件分支将增至 **6-12 个**，方法可能超过 200 行。使用策略模式后，每种格式仍是独立的类，代码量线性增长而非指数膨胀。

1.7.4 模板方法模式 (Template Method Pattern)

1.7.4.1 应用场景：多格式数据导出

系统需要支持三种导出格式，每种格式的导出流程整体相似但细节不同：

导出格式	数据准备	格式转换	文件生成
Excel	查询标注数据	转为二维表格	生成 .xlsx
JSON	查询标注数据	转为 JSON 对象	生成 .json
图片	查询标注数据	渲染论证图示	生成 .png/.svg

共同的流程骨架：

1. 验证任务状态（必须已完成裁决）
2. 查询任务元数据（任务名称、标签版本等）
3. 查询标注数据（命题、关系、裁决结果）
4. 执行格式转换
5. 生成文件并写入磁盘
6. 记录导出日志
7. 更新导出进度

`ExportCoordinator`（实现 `IExportService` 接口）持有 `AbstractExporter` 引用。
`AbstractExporter` 定义 `export()` 模板方法（标记为 `final`，子类不可覆盖），其中步骤 1-3 和 6-7 为通用逻辑（`final` 方法），步骤 4-5 为抽象方法由子类实现。`ExcelExporter`、`JsonExporter`、`ImageExporter` 分别实现各自的 `transform()` 和 `generateFile()`。

1.7.4.2 为什么在这里使用模板方法模式

设计理由	说明
流程骨架固定	所有导出格式都遵循相同的 7 步流程，骨架不应被修改。模板方法的 <code>final export()</code> 从语法层面锁定了流程顺序，子类无法覆盖。
步骤实现不同	第 4 步（格式转换）和第 5 步（文件生成）各格式实现完全不同：Excel 用表格、JSON 用对象树、图片用图形渲染。模板方法将变化点隔离到子类。

设计理由	说明
避免重复代码	步骤 1-3（验证+查询）和 6-7（日志+进度）的通用逻辑在父类中实现，子类直接继承无需重写。消除 3 种格式 × 4 个通用步骤 = 12 处重复代码。
控制流程合规	通过 <code>final</code> 方法锁定骨架流程，防止子类误改执行顺序（如先写日志再生成文件）。所有子类必须遵守父类定义的流程契约。
钩子方法支持扩展	骨架中预留 <code>beforeExport()</code> / <code>afterExport()</code> 钩子方法，子类可选择性覆盖。例如 <code>ImageExporter</code> 在 <code>afterExport()</code> 中释放 <code>Canvas</code> 资源。
与依赖倒转原则配合	<code>Task</code> 依赖 <code>IExportService</code> 接口， <code>ExportCoordinator</code> 实现该接口并委托给 <code>AbstractExporter</code> 。 <code>Task</code> 不直接依赖 <code>ExportLog</code> 具体类，导出实现细节完全隐藏在服务层。

1.7.4.3 如果不用这个模式会怎样

问题	具体影响
重复代码	每个导出类都要重复写验证、查询、日志记录等相同代码。3 种格式就需要 3 份几乎相同的验证+查询逻辑，违反 DRY 原则。修改通用逻辑（如在导出前增加权限校验）需要修改所有导出类。
流程不一致	不同格式导出可能遗漏关键步骤。例如某开发者实现 SVG 导出时忘记记录日志，导致导出历史不完整；或在生成文件前忘记校验任务状态，导致未完成裁决的任务也能导出。
缺乏约束	子类可能错误地改变导出顺序（如先写日志再生成文件），导致数据不一致。没有模板方法的 <code>final</code> 约束，这类错误只能在运行时通过测试发现。
无法统一扩展	新增所有导出类共有的功能（如在导出前发送通知、增加存储空间检查）需要逐一修改每个导出类，维护成本随格式数量线性增长。
违反依赖倒转	若不用模板方法模式，导出逻辑可能直接暴露在 <code>Task</code> 或 <code>ExportCoordinator</code> 中，导致高层模块依赖低层导出细节（如直接依赖 <code>Excel</code> 生成库），破坏分层架构。

具体场景举例：假设需要在所有导出流程中增加"导出前检查存储空间"的步骤。使用模板方法模式时，只需在 `AbstractExporter.export()` 的 `beforeExport()` 之前增加一行调用；不用模板方法模式时，需要在 `ExcelExporter`、`JsonExporter`、`ImageExporter` **三个类中分别修改**，漏改任何一个都会导致部分导出功能缺失校验，且每次新增格式都要手动加上这一检查。

1.7.5 模式对比与总结

设计模式	应用场景	解决的核心问题	若不用该模式的代价
状态模式	任务四阶段生命周期	阶段行为差异 + 状态转换控制	20+ 条件分支，阶段规则散落，非法跳转风险，测试用例指数增长
观察者模式	标注数据 ↔ 图示联动	一对多依赖解耦 + 实时同步	模块强耦合，同步代码遗漏导致界面数据不一致，无法独立测试

设计模式	应用场景	解决的核心问题	若不用该模式的代价
策略模式	多格式文档解析	算法独立封装 + 运行时动态选择	200+ 行条件分支方法，新增格式需改现有代码，测试无法隔离
模板方法模式	多格式数据导出	固定流程骨架 + 步骤实现分离	重复代码，流程遗漏风险，维护成本随格式数线性倍增

二、API设计

1.总览：

本节基于《法律文书标注平台》系统需求与界面设计进行 RESTful API 设计，用于规范前后端的数据交互方式。

系统主要包含以下模块：

- 用户与权限管理
- 文书总库管理
- 标签配置中心
- 任务管理
- 标注工作台
- 裁定工作台
- 结果查看与导出

2. API 设计规范

2.1 基础路径

所有接口统一以：

```
1 | /api
```

作为基础路径。

示例：

```
1 | /api/auth/login
2 | /api/tasks
3 | /api/documents
```

2.2 数据格式

普通接口：

```
1 | Content-Type: application/json
```

文件上传接口：

```
1 Content-Type: multipart/form-data
```

2.3 身份认证

除登录接口外，其余接口均需携带 Token：

```
1 Authorization: Bearer {token}
```

2.4 通用响应格式

成功响应

```
1 {
2   "code": 200,
3   "message": "success",
4   "data": {}
5 }
```

失败响应

```
1 {
2   "code": 400,
3   "message": "参数错误",
4   "data": null
5 }
```

2.5 统一错误码定义

错误码	含义
200	请求成功
400	请求参数错误
401	未登录或 Token 无效
403	权限不足
404	资源不存在
409	数据冲突
500	服务器内部错误

3. 用户与权限模块 API

对应页面：

- P3 用户管理

3.1 用户登录

接口说明

用户输入账号密码后登录系统。

登录成功后返回 JWT Token 与用户角色信息。

URL

```
1 | POST /api/auth/login
```

HTTP Method

```
1 | POST
```

请求参数

参数名	类型	必填	说明
username	string	是	用户账号
password	string	是	用户密码

请求示例

```
1 | {
2 |   "username": "admin",
3 |   "password": "123456"
4 | }
```

成功响应

```
1 | {
2 |   "code": 200,
3 |   "message": "登录成功",
4 |   "data": {
5 |     "token": "jwt-token-string",
6 |     "user": {
7 |       "id": 1,
8 |       "username": "admin",
9 |       "role": "admin"
10 |    }
11 |  }
12 | }
```

失败响应

```
1 {
2   "code": 401,
3   "message": "用户名或密码错误",
4   "data": null
5 }
```

错误码定义

错误码	说明
400	参数为空
401	登录失败
500	服务器异常

3.2 获取当前用户信息

接口说明

获取当前登录用户的基本信息与角色权限。

URL

```
1 GET /api/users/me
```

请求头

```
1 Authorization: Bearer {token}
```

成功响应

```
1 {
2   "code": 200,
3   "message": "success",
4   "data": {
5     "id": 1,
6     "username": "admin",
7     "role": "admin"
8   }
9 }
```

失败响应

```
1 {
2   "code": 401,
3   "message": "Token 无效",
4   "data": null
5 }
```

错误码定义

错误码	说明
401	Token 无效
500	服务器异常

3.3 新增用户

接口说明

管理员新增系统用户。

对应页面：

- P3 用户管理

URL

```
1 | POST /api/users
```

HTTP Method

```
1 | POST
```

请求参数

参数名	类型	必填	说明
username	string	是	登录账号
realName	string	是	用户姓名
password	string	是	用户密码
role	string	是	admin/creator/annotator/reviewer
canCreateTask	boolean	是	是否允许创建任务

请求示例

```
1 | {
2 |   "username": "user01",
3 |   "realName": "张三",
4 |   "password": "123456",
5 |   "role": "annotator",
6 |   "canCreateTask": false
7 | }
```

成功响应

```
1 {
2   "code": 200,
3   "message": "用户创建成功",
4   "data": {
5     "userId": 12
6   }
7 }
```

失败响应

```
1 {
2   "code": 409,
3   "message": "用户名已存在",
4   "data": null
5 }
```

错误码定义

错误码	说明
400	参数缺失
403	权限不足
409	用户重复
500	创建失败

3.4 编辑用户

接口说明

修改用户信息。

URL

```
1 PUT /api/users/{userId}
```

请求参数

参数名	类型	必填	说明
realName	string	否	用户姓名
password	string	否	新密码
canCreateTask	boolean	否	是否允许创建任务

成功响应

```
1 {
2   "code": 200,
3   "message": "修改成功",
4   "data": null
5 }
```

失败响应

```
1 {
2   "code": 404,
3   "message": "用户不存在",
4   "data": null
5 }
```

错误码定义

错误码	说明
400	参数非法
403	无权限
404	用户不存在
500	修改失败

3.5 删除用户

接口说明

删除指定用户。

URL

```
1 DELETE /api/users/{userId}
```

请求参数

参数名	类型	必填	说明
userId	int	是	用户ID

成功响应

```
1 {
2   "code": 200,
3   "message": "删除成功",
4   "data": null
5 }
```

失败响应

```
1 {
2   "code": 403,
3   "message": "权限不足",
4   "data": null
5 }
```

错误码定义

错误码	说明
403	权限不足
404	用户不存在
500	删除失败

4. 文书总库模块 API

对应页面：

- P1 文书总库

4.1 批量上传文书

接口说明

上传 PDF、Word、TXT 文书文件。

系统自动进行：

- 文件格式校验
- 哈希去重校验

URL

```
1 | POST /api/documents/upload
```

请求类型

```
1 | multipart/form-data
```

请求参数

参数名	类型	必填	说明
files	File[]	是	上传文件列表

成功响应

```
1 {
2   "code": 200,
3   "message": "上传成功",
4   "data": {
5     "count": 5
6   }
7 }
```

失败响应

```
1 {
2   "code": 400,
3   "message": "文件格式错误",
4   "data": null
5 }
```

错误码定义

错误码	说明
400	文件格式错误
409	文件重复上传
500	上传失败

4.2 获取文书列表

接口说明

获取文书库列表。

支持：

- 文书ID筛选
- 标题筛选
- 上传时间筛选

URL

```
1 GET /api/documents
```

Query 参数

参数名	类型	必填	说明
documentId	string	否	文书ID
title	string	否	文书标题
uploadDate	string	否	上传日期

参数名	类型	必填	说明
page	int	否	页码
size	int	否	每页数量

成功响应

```
1  {
2    "code": 200,
3    "message": "success",
4    "data": {
5      "total": 100,
6      "list": [
7        {
8          "documentId": "w001",
9          "title": "合同纠纷案",
10         "uploadDate": "2026-05-04"
11       }
12     ]
13   }
14 }
```

失败响应

```
1  {
2    "code": 500,
3    "message": "查询失败",
4    "data": null
5  }
```

错误码定义

错误码	说明
400	参数错误
500	查询失败

5. 配置中心模块 API

对应页面：

- P2 配置中心

5.1 创建指南版本

接口说明

创建新的标签配置版本。

包含：

- 一级标签
- 二级标签
- 关系类型

URL

```
1 | POST /api/configs/versions
```

请求参数

```
1 | {
2 |   "versionName": "v1.0",
3 |   "primaryTags": [
4 |     {
5 |       "name": "一般事实判断",
6 |       "shortName": "GF"
7 |     }
8 |   ],
9 |   "secondaryTags": [
10 |    {
11 |      "name": "法律条文",
12 |      "shortName": "GM-L"
13 |    }
14 |  ],
15 |   "relationTypes": [
16 |     {
17 |       "name": "支持",
18 |       "shortName": "S"
19 |     }
20 |   ]
21 | }
```

成功响应

```
1 | {
2 |   "code": 200,
3 |   "message": "配置保存成功",
4 |   "data": {
5 |     "configId": 1
6 |   }
7 | }
```

失败响应

```
1 {
2   "code": 400,
3   "message": "版本名称不能为空",
4   "data": null
5 }
```

错误码定义

错误码	说明
400	参数错误
403	权限不足
500	保存失败

6. 任务管理模块 API

对应页面：

- P4
- P5
- P6

6.1 创建任务

接口说明

创建新的法律文书标注任务。

对应作业中的：

发布需求

URL

```
1 POST /api/tasks
```

请求参数

参数名	类型	必填	说明
taskName	string	是	任务名称
configId	int	是	标签配置ID
annotatorIds	int[]	是	标注员列表
reviewerId	int	是	裁定者ID
dataSourceType	string	是	text/upload/library

参数名	类型	必填	说明
content	string	否	文本内容
documentIds	string[]	否	文书ID列表

请求示例

```
1 {
2   "taskName": "合同法标注任务",
3   "configId": 1,
4   "annotatorIds": [2,3],
5   "reviewerId": 5,
6   "dataSourceType": "library",
7   "documentIds": ["w001","w002"]
8 }
```

成功响应

```
1 {
2   "code": 200,
3   "message": "任务创建成功",
4   "data": {
5     "taskId": 1001
6   }
7 }
```

失败响应

```
1 {
2   "code": 400,
3   "message": "标注员不能为空",
4   "data": null
5 }
```

错误码定义

错误码	说明
400	参数不完整
403	无权限
404	配置不存在
500	创建失败

6.2 获取任务列表

接口说明

获取任务列表。

支持：

- 角色筛选
- 状态筛选
- 关键词搜索

对应作业中的：

浏览需求列表

URL

```
1 GET /api/tasks
```

Query 参数

参数名	类型	必填	说明
role	string	否	creator/annotator/reviewer
status	string	否	标注中/裁决中/已完成
keyword	string	否	搜索关键词
page	int	否	页码
size	int	否	每页数量

成功响应

```
1 {
2   "code": 200,
3   "message": "success",
4   "data": {
5     "total": 20,
6     "list": [
7       {
8         "taskId": 1001,
9         "taskName": "合同法标注",
10        "status": "标注中"
11      }
12    ]
13  }
14 }
```

错误码定义

错误码	说明
400	参数错误
500	查询失败

6.3 获取任务详情

接口说明

获取指定任务详细信息。

对应作业中的：

查看订单详情

URL

```
1 GET /api/tasks/{taskId}
```

请求参数

参数名	类型	必填	说明
taskId	int	是	任务ID

成功响应

```
1 {
2   "code": 200,
3   "message": "success",
4   "data": {
5     "taskId": 1001,
6     "taskName": "合同法任务",
7     "status": "标注中",
8     "annotators": [],
9     "reviewer": {}
10  }
11 }
```

错误码定义

错误码	说明
404	任务不存在
500	查询失败

7. 标注工作台 API

对应页面：

- P8 标注工作台
- P11 数据选择页

7.1 获取待标注数据列表

URL

```
1 | GET /api/tasks/{taskId}/items
```

Query 参数

参数名	类型	必填	说明
status	string	否	待标注/已标注

成功响应

```
1 | {
2 |   "code": 200,
3 |   "message": "success",
4 |   "data": [
5 |     {
6 |       "dataId": "D001",
7 |       "status": "待标注"
8 |     }
9 |   ]
10 | }
```

错误码定义

错误码	说明
404	任务不存在
500	查询失败

7.2 获取标注详情

URL

```
1 | GET /api/tasks/{taskId}/items/{dataId}
```

成功响应

```
1 {
2   "code": 200,
3   "message": "success",
4   "data": {
5     "content": "依法成立的合同，自成立时生效。",
6     "propositions": [],
7     "relations": []
8   }
9 }
```

错误码定义

错误码	说明
404	数据不存在
500	查询失败

7.3 提交标注结果

接口说明

提交或暂存标注结果。

对应作业中的：

提交评价

URL

```
1 POST /api/annotations/submit
```

请求参数

```
1 {
2   "taskId": 1001,
3   "dataId": "D001",
4   "propositions": [],
5   "relations": [],
6   "isDraft": false
7 }
```

成功响应

```
1 {
2   "code": 200,
3   "message": "提交成功",
4   "data": null
5 }
```

失败响应

```
1 {
2   "code": 400,
3   "message": "关系非法",
4   "data": null
5 }
```

错误码定义

错误码	说明
400	关系结构非法
403	无标注权限
404	数据不存在
500	提交失败

8. 裁定模块 API

对应页面：

- P9 裁定界面

8.1 获取裁定数据

URL

```
1 GET /api/reviews/{taskId}
```

成功响应

```
1 {
2   "code": 200,
3   "message": "success",
4   "data": {
5     "annotatorResults": []
6   }
7 }
```

错误码定义

错误码	说明
403	无裁定权限
404	任务不存在

8.2 全部采纳

URL

```
1 | POST /api/reviews/adopt
```

请求参数

参数名	类型	必填	说明
taskId	int	是	任务ID
annotatorId	int	是	被采纳标注员ID

成功响应

```
1 | {
2 |   "code": 200,
3 |   "message": "裁定完成",
4 |   "data": null
5 | }
```

错误码定义

错误码	说明
403	无裁定权限
404	标注结果不存在
500	裁定失败

9. 结果查看与导出 API

对应页面：

- P7
- P10

9.1 获取结果列表

URL

```
1 | GET /api/tasks/{taskId}/results
```

成功响应

```
1 | {
2 |   "code": 200,
3 |   "message": "success",
4 |   "data": []
5 | }
```

错误码定义

错误码	说明
404	任务不存在
500	查询失败

9.2 导出结果

接口说明

导出：

- PNG
- SVG
- JSON
- XLSX
- ZIP

格式结果。

仅任务创建者与裁定者允许导出。

URL

```
1 | GET /api/tasks/{taskId}/export
```

Query 参数

参数名	类型	必填	说明
format	string	是	json/xlsx/png/svg/zip

成功响应

```
1 | {
2 |   "code": 200,
3 |   "message": "导出成功",
4 |   "data": {
5 |     "downloadUrl": "/download/task1001.zip"
6 |   }
7 | }
```

失败响应

```
1 | {
2 |   "code": 403,
3 |   "message": "无导出权限",
4 |   "data": null
5 | }
```

错误码定义

错误码	说明
400	导出格式非法
403	无导出权限
404	任务不存在
500	导出失败

10. AI 辅助实验分析

10.1 接口命名问题

AI 初版生成存在：

```
1  /getTaskList
2  /updateUser
```

等不规范命名。

修正后统一采用 RESTful 风格：

```
1  GET  /api/tasks
2  PUT  /api/users/{id}
```

10.2 缺少错误处理

AI 初版仅包含成功响应。

未考虑：

- Token 失效
- 权限不足
- 参数错误
- 数据不存在

等情况。

因此增加：

- 统一错误码
- 失败响应结构

10.3 参数校验不完整

AI 初版未校验:

- 标注员不能为空
- 文件格式是否合法
- 关系是否合法

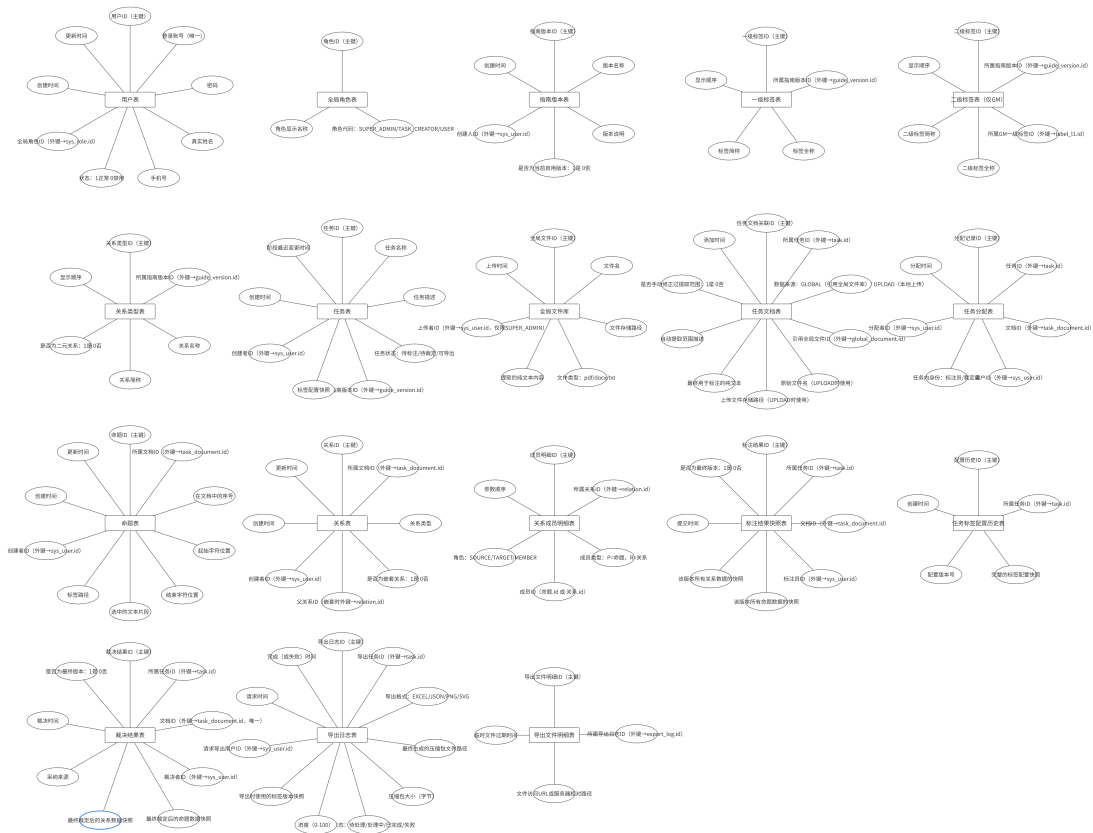
修正后增加：

- 必填字段校验
- 文件类型校验
- 关系结构校验

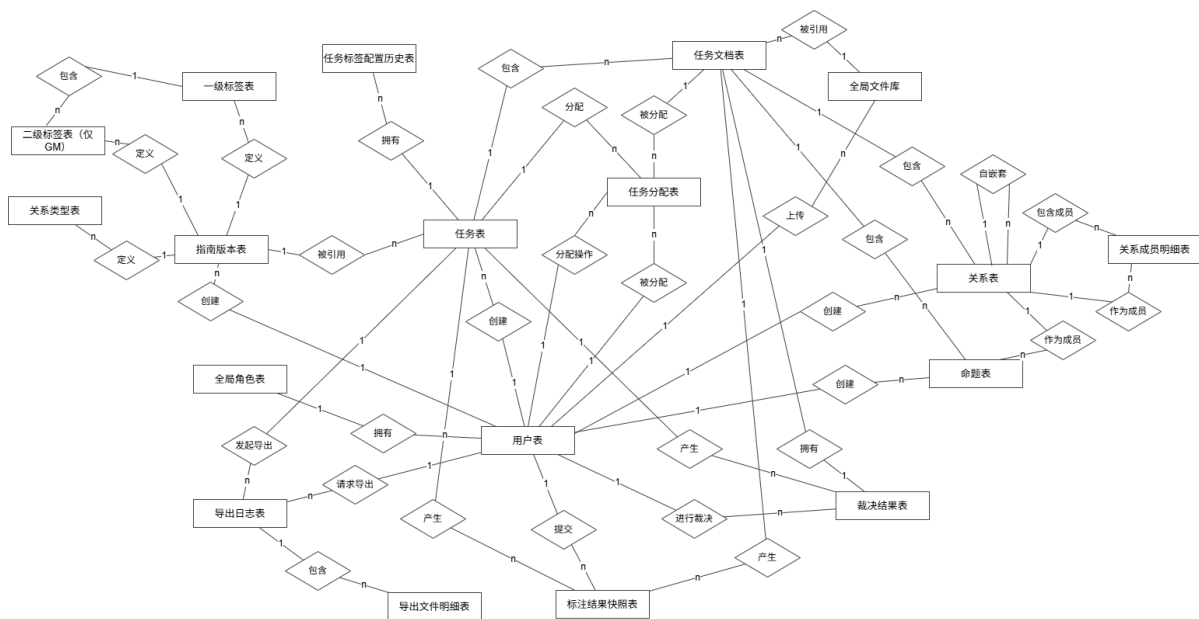
三、数据库设计

3.1 ER圖

1.实体属性



2.实体间关系



各实体间关系

序号	源实体	目标实体	关系描述	基数	外键位置	备注
1	sys_user	sys_role	拥有	N:1	sys_user.role_id → sys_role.id	一个用户只有一个全局角色
2	sys_user	guide_version	创建	1:N	guide_version.created_by → sys_user.id	超级管理员创建指南版本
3	sys_user	global_document	上传	1:N	global_document.uploaded_by_id → sys_user.id	超级管理员上传全局文件
4	sys_user	task	创建	1:N	task.creator_id → sys_user.id	任务创建者创建任务
5	sys_user	proposition	创建	1:N	proposition.created_by → sys_user.id	标注员创建命题
6	sys_user	relation	创建	1:N	relation.created_by → sys_user.id	标注员创建关系
7	sys_user	annotation	提交	1:N	annotation.user_id → sys_user.id	标注员提交标注快照
8	sys_user	arbitration	执行	1:N	arbitration.arbitrator_id → sys_user.id	裁决者执行裁决
9	sys_user	export_log	请求	1:N	export_log.requested_by → sys_user.id	用户请求导出

序号	源实体	目标实体	关系描述	基数	外键位置	备注
10	sys_user	task_assignment	被分配	1:N	task_assignment.user_id → sys_user.id	用户被分配为标注员/裁决者
11	sys_user	task_assignment	操作分配	1:N	task_assignment.assigned_by → sys_user.id	任务创建者分配人员
12	guide_version	label_l1	定义	1:N	label_l1.guide_version_id → guide_version.id	版本包含一级标签
13	guide_version	label_l2	定义	1:N	label_l2.guide_version_id → guide_version.id	版本包含二级标签
14	guide_version	relation_type	定义	1:N	relation_type.guide_version_id → guide_version.id	版本包含关系类型
15	guide_version	task	被引用	1:N	task.guide_version_id → guide_version.id	一个版本可被多个任务使用
16	label_l1	label_l2	包含	1:N	label_l2.parent_l1_id → label_l1.id	GM标签包含二级标签
17	global_document	task_document	被引用	1:N	task_document.global_doc_id → global_document.id	全局文件被多个任务引用
18	task	task_document	包含	1:N	task_document.task_id → task.id	任务包含多个数据条目
19	task	task_assignment	分配	1:N	task_assignment.task_id → task.id	任务有多条分配记录
20	task	annotation	产生快照	1:N	annotation.task_id → task.id	任务有多个标注结果快照
21	task	arbitration	产生	1:N	arbitration.task_id → task.id	任务有多个文档的裁决结果
22	task	export_log	发起导出	1:N	export_log.task_id → task.id	任务可被多次导出
23	task	label_config	拥有配置历史	1:N	label_config.task_id → task.id	任务可有多配置版本

序号	源实体	目标实体	关系描述	基数	外键位置	备注
24	task_document	task_assignment	被分配	1:N	task_assignment.document_id → task_document.id	数据条目可分配给多个用户
25	task_document	proposition	包含	1:N	proposition.document_id → task_document.id	文档包含多个命题
26	task_document	relation	包含	1:N	relation.document_id → task_document.id	文档包含多个关系
27	task_document	annotation	产生快照	1:N	annotation.document_id → task_document.id	文档有多个标注员提交的快照
28	task_document	arbitration	拥有	1:1	arbitration.document_id → task_document.id (UNIQUE)	每个文档仅一条最终裁决
29	relation	relation	自嵌套	1:N	relation.parent_relation_id → relation.id	关系可包含子关系
30	relation	relation_member	包含成员	1:N	relation_member.relation_id → relation.id	关系对应多条成员记录
31	proposition	relation_member	作为成员	1:N	relation_member.member_id (member_type='P')	命题被多个关系引用
32	relation	relation_member	作为成员	1:N	relation_member.member_id (member_type='R')	关系被多个父关系引用
33	export_log	export_file	包含文件	1:N	export_file.export_log_id → export_log.id	一次导出可生成多个文件

3.2 建表SQL

```
1  -- ===== 用户权限模块 =====
2
3  -- 用户表
4  CREATE TABLE `sys_user` (
5      `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '用户ID（主键）',
6      `username` VARCHAR(50) NOT NULL COMMENT '登录账号（唯一）',
7      `password_hash` VARCHAR(255) NOT NULL COMMENT '密码',
8      `real_name` VARCHAR(100) NOT NULL COMMENT '真实姓名',
9      `phone` VARCHAR(20) DEFAULT NULL COMMENT '手机号',
10     `status` TINYINT NOT NULL DEFAULT 1 COMMENT '状态: 1正常 0禁用',
11     `role_id` INT UNSIGNED NOT NULL COMMENT '全局角色ID（外键→sys_role.id）',
12     `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
```

```

13     `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
14     PRIMARY KEY (`id`),
15     UNIQUE KEY `uk_username` (`username`),
16     KEY `idx_status` (`status`),
17     KEY `idx_role` (`role_id`),
18     CONSTRAINT `fk_user_role` FOREIGN KEY (`role_id`) REFERENCES `sys_role`
(`id`) ON DELETE RESTRICT
19 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';
20
21 -- 全局角色表
22 CREATE TABLE `sys_role` (
23     `id` INT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '角色ID（主键）',
24     `role_code` VARCHAR(30) NOT NULL COMMENT '角色代码：
SUPER_ADMIN/TASK_CREATOR/USER',
25     `role_name` VARCHAR(50) NOT NULL COMMENT '角色显示名称',
26     PRIMARY KEY (`id`),
27     UNIQUE KEY `uk_role_code` (`role_code`)
28 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='全局角色表';
29
30 -- ===== 配置中心模块 =====
31
32 -- 指南版本表
33 CREATE TABLE `guide_version` (
34     `id` INT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '指南版本ID（主键）',
35     `version_name` VARCHAR(100) NOT NULL COMMENT '版本名称',
36     `description` TEXT COMMENT '版本说明',
37     `is_active` TINYINT NOT NULL DEFAULT 0 COMMENT '是否为当前启用版本：1是 0
否',
38     `created_by` BIGINT UNSIGNED NOT NULL COMMENT '创建人ID（外键
→sys_user.id)',
39     `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
40     PRIMARY KEY (`id`),
41     KEY `idx_active` (`is_active`),
42     CONSTRAINT `fk_guide_version_user` FOREIGN KEY (`created_by`)
REFERENCES `sys_user` (`id`) ON DELETE RESTRICT
43 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='指南版本表';
44
45 -- 一级标签表
46 CREATE TABLE `label_11` (
47     `id` INT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '一级标签ID（主键）',
48     `guide_version_id` INT UNSIGNED NOT NULL COMMENT '所属指南版本ID（外键
→guide_version.id)',
49     `name` VARCHAR(50) NOT NULL COMMENT '标签全称',
50     `abbr` VARCHAR(10) NOT NULL COMMENT '标签简称',
51     `sort_order` INT NOT NULL DEFAULT 0 COMMENT '显示顺序',
52     PRIMARY KEY (`id`),
53     UNIQUE KEY `uk_version_abbr` (`guide_version_id`, `abbr`),
54     CONSTRAINT `fk_label_11_version` FOREIGN KEY (`guide_version_id`)
REFERENCES `guide_version` (`id`) ON DELETE CASCADE
55 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='一级标签表';
56
57 -- 二级标签表
58 CREATE TABLE `label_12` (
59     `id` INT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '二级标签ID（主键）',

```

```

60     `guide_version_id` INT UNSIGNED NOT NULL COMMENT '所属指南版本ID（外键
    →guide_version.id）',
61     `parent_l1_id` INT UNSIGNED NOT NULL COMMENT '所属GM一级标签ID（外键
    →label_l1.id）',
62     `name` VARCHAR(50) NOT NULL COMMENT '二级标签全称',
63     `abbr` VARCHAR(20) NOT NULL COMMENT '二级标签简称',
64     `sort_order` INT NOT NULL DEFAULT 0 COMMENT '显示顺序',
65     PRIMARY KEY (`id`),
66     KEY `idx_parent` (`parent_l1_id`),
67     CONSTRAINT `fk_label_l2_version` FOREIGN KEY (`guide_version_id`)
    REFERENCES `guide_version` (`id`) ON DELETE CASCADE,
68     CONSTRAINT `fk_label_l2_parent` FOREIGN KEY (`parent_l1_id`) REFERENCES
    `label_l1` (`id`) ON DELETE CASCADE
69 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='二级标签表（仅GM）';
70
71 -- 关系类型表
72 CREATE TABLE `relation_type` (
73     `id` INT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '关系类型ID（主键）',
74     `guide_version_id` INT UNSIGNED NOT NULL COMMENT '所属指南版本ID（外键
    →guide_version.id）',
75     `name` VARCHAR(50) NOT NULL COMMENT '关系名称',
76     `abbr` CHAR(1) NOT NULL COMMENT '关系简称',
77     `is_binary` TINYINT NOT NULL DEFAULT 1 COMMENT '是否为二元关系：1是 0否',
78     `sort_order` INT NOT NULL DEFAULT 0 COMMENT '显示顺序',
79     PRIMARY KEY (`id`),
80     UNIQUE KEY `uk_version_abbr` (`guide_version_id`, `abbr`),
81     CONSTRAINT `fk_relation_type_version` FOREIGN KEY (`guide_version_id`)
    REFERENCES `guide_version` (`id`) ON DELETE CASCADE
82 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='关系类型表';
83
84 -- ===== 任务管理模块 =====
85
86 -- 任务表
87 CREATE TABLE `task` (
88     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '任务ID（主键）',
89     `title` VARCHAR(200) NOT NULL COMMENT '任务名称',
90     `description` TEXT COMMENT '任务描述',
91     `status` VARCHAR(20) NOT NULL DEFAULT '待标注' COMMENT '任务状态：待标注/待
    裁定/可导出',
92     `guide_version_id` INT UNSIGNED NOT NULL COMMENT '使用的指南版本ID（外键
    →guide_version.id）',
93     `label_config_snapshot` JSON NOT NULL COMMENT '标签配置快照',
94     `creator_id` BIGINT UNSIGNED NOT NULL COMMENT '创建者ID（外键
    →sys_user.id）',
95     `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
    间',
96     `stage_changed_at` DATETIME DEFAULT NULL COMMENT '阶段最近变更时间',
97     PRIMARY KEY (`id`),
98     KEY `idx_creator` (`creator_id`),
99     KEY `idx_status` (`status`),
100    KEY `idx_guide_version` (`guide_version_id`),
101    CONSTRAINT `fk_task_creator` FOREIGN KEY (`creator_id`) REFERENCES
    `sys_user` (`id`) ON DELETE RESTRICT,
102    CONSTRAINT `fk_task_guide_version` FOREIGN KEY (`guide_version_id`)
    REFERENCES `guide_version` (`id`) ON DELETE RESTRICT
103 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='任务表';

```

```

104
105 -- 全局文件表
106 CREATE TABLE `global_document` (
107     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '全局文件ID (主键)',
108     `file_name` VARCHAR(255) NOT NULL COMMENT '文件名',
109     `file_path` VARCHAR(500) NOT NULL COMMENT '文件存储路径',
110     `file_type` VARCHAR(10) NOT NULL COMMENT '文件类型: pdf/docx/txt',
111     `extracted_text` LONGTEXT NOT NULL COMMENT '提取的纯文本内容',
112     `uploaded_by_id` BIGINT UNSIGNED NOT NULL COMMENT '上传者ID (外键
→sys_user.id, 仅限SUPER_ADMIN)',
113     `uploaded_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '上传
时间',
114     PRIMARY KEY (`id`),
115     KEY `idx_uploader` (`uploaded_by_id`),
116     KEY `idx_file_name` (`file_name`(100)),
117     CONSTRAINT `fk_global_doc_user` FOREIGN KEY (`uploaded_by_id`)
REFERENCES `sys_user` (`id`) ON DELETE RESTRICT
118 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='全局文件库';
119
120 -- 任务文档表
121 CREATE TABLE `task_document` (
122     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '任务文档关联ID (主
键)',
123     `task_id` BIGINT UNSIGNED NOT NULL COMMENT '所属任务ID (外键→task.id)',
124     `source_type` ENUM('GLOBAL', 'UPLOAD') NOT NULL COMMENT '数据来源: GLOBAL
(引用全局文件库) / UPLOAD (本地上传)',
125     `global_doc_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '引用全局文件ID (外键
→global_document.id)',
126     `file_name` VARCHAR(255) DEFAULT NULL COMMENT '原始文件名 (UPLOAD时使用)',
127     `file_path` VARCHAR(500) DEFAULT NULL COMMENT '上传文件存储路径 (UPLOAD时使用)',
128     `extracted_text` LONGTEXT NOT NULL COMMENT '最终用于标注的纯文本',
129     `extracted_range` VARCHAR(100) DEFAULT NULL COMMENT '自动提取范围描述',
130     `is_manually_corrected` TINYINT NOT NULL DEFAULT 0 COMMENT '是否手动修正
过提取范围: 1是 0否',
131     `uploaded_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '添加
时间',
132     PRIMARY KEY (`id`),
133     KEY `idx_task` (`task_id`),
134     KEY `idx_global` (`global_doc_id`),
135     CONSTRAINT `fk_td_task` FOREIGN KEY (`task_id`) REFERENCES `task`
(`id`) ON DELETE CASCADE,
136     CONSTRAINT `fk_td_global` FOREIGN KEY (`global_doc_id`) REFERENCES
`global_document` (`id`) ON DELETE SET NULL
137 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='任务文档表';
138
139 -- 任务分配表
140 CREATE TABLE `task_assignment` (
141     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '分配记录ID (主键)',
142     `task_id` BIGINT UNSIGNED NOT NULL COMMENT '任务ID (外键→task.id)',
143     `document_id` BIGINT UNSIGNED NOT NULL COMMENT '文档ID (外键
→task_document.id)',
144     `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID (外键→sys_user.id)',
145     `role_in_task` VARCHAR(20) NOT NULL COMMENT '任务内身份: 标注员/裁定者',

```

```

146     `assigned_by` BIGINT UNSIGNED NOT NULL COMMENT '分配者ID（外键
→sys_user.id）',
147     `assigned_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '分配
时间',
148     PRIMARY KEY (`id`),
149     UNIQUE KEY `uk_task_doc_user_role` (`task_id`, `document_id`,
`user_id`, `role_in_task`),
150     KEY `idx_user` (`user_id`),
151     KEY `idx_task_role` (`task_id`, `role_in_task`),
152     CONSTRAINT `fk_ta_task` FOREIGN KEY (`task_id`) REFERENCES `task`
(`id`) ON DELETE CASCADE,
153     CONSTRAINT `fk_ta_document` FOREIGN KEY (`document_id`) REFERENCES
`task_document` (`id`) ON DELETE CASCADE,
154     CONSTRAINT `fk_ta_user` FOREIGN KEY (`user_id`) REFERENCES `sys_user`
(`id`) ON DELETE RESTRICT,
155     CONSTRAINT `fk_ta_assigned_by` FOREIGN KEY (`assigned_by`) REFERENCES
`sys_user` (`id`) ON DELETE RESTRICT
156 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='任务分配表';
157
158 -- ===== 标注核心模块 =====
159
160 -- 命题表
161 CREATE TABLE `proposition` (
162     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '命题ID（主键）',
163     `document_id` BIGINT UNSIGNED NOT NULL COMMENT '所属文档ID（外键
→task_document.id）',
164     `sequence_no` INT NOT NULL COMMENT '在文档中的序号',
165     `start_pos` INT NOT NULL COMMENT '起始字符位置',
166     `end_pos` INT NOT NULL COMMENT '结束字符位置',
167     `selected_text` TEXT NOT NULL COMMENT '选中的文本片段',
168     `label_path` VARCHAR(50) NOT NULL COMMENT '标签路径',
169     `created_by` BIGINT UNSIGNED NOT NULL COMMENT '创建者ID（外键
→sys_user.id）',
170     `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
171     `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
172     PRIMARY KEY (`id`),
173     KEY `idx_document` (`document_id`),
174     KEY `idx_sequence` (`document_id`, `sequence_no`),
175     CONSTRAINT `fk_prop_doc` FOREIGN KEY (`document_id`) REFERENCES
`task_document` (`id`) ON DELETE CASCADE,
176     CONSTRAINT `fk_prop_user` FOREIGN KEY (`created_by`) REFERENCES
`sys_user` (`id`) ON DELETE RESTRICT
177 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='命题表';
178
179 -- 关系表
180 CREATE TABLE `relation` (
181     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '关系ID（主键）',
182     `document_id` BIGINT UNSIGNED NOT NULL COMMENT '所属文档ID（外键
→task_document.id）',
183     `relation_type` CHAR(1) NOT NULL COMMENT '关系类型',
184     `is_nested` TINYINT NOT NULL DEFAULT 0 COMMENT '是否为嵌套关系: 1是 0否',
185     `parent_relation_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '父关系ID（嵌套
时外键→relation.id）',

```

```

186     `created_by` BIGINT UNSIGNED NOT NULL COMMENT '创建者ID（外键
→sys_user.id）',
187     `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
188     `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
189     PRIMARY KEY (`id`),
190     KEY `idx_document` (`document_id`),
191     KEY `idx_parent` (`parent_relation_id`),
192     CONSTRAINT `fk_rel_doc` FOREIGN KEY (`document_id`) REFERENCES
`task_document` (`id`) ON DELETE CASCADE,
193     CONSTRAINT `fk_rel_parent` FOREIGN KEY (`parent_relation_id`)
REFERENCES `relation` (`id`) ON DELETE CASCADE,
194     CONSTRAINT `fk_rel_user` FOREIGN KEY (`created_by`) REFERENCES
`sys_user` (`id`) ON DELETE RESTRICT
195 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='关系表';
196
197 -- 关系成员明细表
198 CREATE TABLE `relation_member` (
199     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '成员明细ID（主键）',
200     `relation_id` BIGINT UNSIGNED NOT NULL COMMENT '所属关系ID（外键
→relation.id）',
201     `member_type` ENUM('P', 'R') NOT NULL COMMENT '成员类型：P=命题，R=关系',
202     `member_id` BIGINT UNSIGNED NOT NULL COMMENT '成员ID（命题.id 或 关
系.id）',
203     `member_role` VARCHAR(10) NOT NULL DEFAULT 'MEMBER' COMMENT '角色：
SOURCE/TARGET/MEMBER',
204     `member_order` INT NOT NULL DEFAULT 0 COMMENT '参数顺序',
205     PRIMARY KEY (`id`),
206     KEY `idx_relation` (`relation_id`),
207     KEY `idx_member` (`member_type`, `member_id`),
208     CONSTRAINT `fk_rm_relation` FOREIGN KEY (`relation_id`) REFERENCES
`relation` (`id`) ON DELETE CASCADE
209 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='关系成员明细表';
210
211 -- 标注结果快照表
212 CREATE TABLE `annotation` (
213     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '标注结果ID（主键）',
214     `task_id` BIGINT UNSIGNED NOT NULL COMMENT '所属任务ID（外键→task.id）',
215     `document_id` BIGINT UNSIGNED NOT NULL COMMENT '文档ID（外键
→task_document.id）',
216     `user_id` BIGINT UNSIGNED NOT NULL COMMENT '标注员ID（外键
→sys_user.id）',
217     `proposition_data` JSON NOT NULL COMMENT '该版本所有命题数据的快照',
218     `relation_data` JSON NOT NULL COMMENT '该版本所有关系数据的快照',
219     `submitted_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '提交
时间',
220     `is_final` TINYINT NOT NULL DEFAULT 0 COMMENT '是否为最终版本：1是 0否',
221     PRIMARY KEY (`id`),
222     UNIQUE KEY `uk_doc_user` (`document_id`, `user_id`),
223     KEY `idx_task` (`task_id`),
224     KEY `idx_submitted` (`submitted_at`),
225     CONSTRAINT `fk_ann_task` FOREIGN KEY (`task_id`) REFERENCES `task`
(`id`) ON DELETE CASCADE,
226     CONSTRAINT `fk_ann_doc` FOREIGN KEY (`document_id`) REFERENCES
`task_document` (`id`) ON DELETE CASCADE,

```

```

227     CONSTRAINT `fk_ann_user` FOREIGN KEY (`user_id`) REFERENCES `sys_user`
228     (`id`) ON DELETE RESTRICT
229 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='标注结果快照表';
230
231 -- 任务标签配置历史表
232 CREATE TABLE `label_config` (
233     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '配置历史ID (主键)',
234     `task_id` BIGINT UNSIGNED NOT NULL COMMENT '所属任务ID (外键→task.id)',
235     `config_json` JSON NOT NULL COMMENT '完整的标签配置快照',
236     `version` INT NOT NULL COMMENT '配置版本号',
237     `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
238 间',
239     PRIMARY KEY (`id`),
240     KEY `idx_task` (`task_id`),
241     CONSTRAINT `fk_lcfg_task` FOREIGN KEY (`task_id`) REFERENCES `task`
242     (`id`) ON DELETE CASCADE
243 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='任务标签配置历史表';
244
245 -- ===== 冲突裁决与导出模块 =====
246
247 -- 裁决结果表
248 CREATE TABLE `arbitration` (
249     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '裁决结果ID (主键)',
250     `task_id` BIGINT UNSIGNED NOT NULL COMMENT '所属任务ID (外键→task.id)',
251     `document_id` BIGINT UNSIGNED NOT NULL COMMENT '文档ID (外键
252 →task.document.id, 唯一)',
253     `arbitrator_id` BIGINT UNSIGNED NOT NULL COMMENT '裁决者ID (外键
254 →sys_user.id)',
255     `final_proposition_data` JSON NOT NULL COMMENT '最终裁定后的命题数据快照',
256     `final_relation_data` JSON NOT NULL COMMENT '最终裁定后的关系数据快照',
257     `adopted_from` VARCHAR(10) NOT NULL COMMENT '采纳来源',
258     `arbitrated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '裁
259 决时间',
260     `is_final` TINYINT NOT NULL DEFAULT 1 COMMENT '是否为最终版本: 1是 0否',
261     PRIMARY KEY (`id`),
262     UNIQUE KEY `uk_document` (`document_id`),
263     KEY `idx_task` (`task_id`),
264     KEY `idx_arbitrator` (`arbitrator_id`),
265     CONSTRAINT `fk_arb_task` FOREIGN KEY (`task_id`) REFERENCES `task`
266     (`id`) ON DELETE CASCADE,
267     CONSTRAINT `fk_arb_doc` FOREIGN KEY (`document_id`) REFERENCES
268     `task_document` (`id`) ON DELETE CASCADE,
269     CONSTRAINT `fk_arb_user` FOREIGN KEY (`arbitrator_id`) REFERENCES
270     `sys_user` (`id`) ON DELETE RESTRICT
271 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='裁决结果表';
272
273 -- 导出日志表
274 CREATE TABLE `export_log` (
275     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '导出日志ID (主键)',
276     `task_id` BIGINT UNSIGNED NOT NULL COMMENT '导出任务ID (外键→task.id)',
277     `export_type` VARCHAR(20) NOT NULL COMMENT '导出格式:
278 EXCEL/JSON/PNG/SVG',
279     `file_path` VARCHAR(500) DEFAULT NULL COMMENT '最终生成的压缩包文件路径',
280     `file_size` BIGINT DEFAULT NULL COMMENT '压缩包大小 (字节)',
281     `status` VARCHAR(20) NOT NULL DEFAULT 'PENDING' COMMENT '导出状态: 待处理/
282 处理中/已完成/失败',

```

```
272     `progress` INT NOT NULL DEFAULT 0 COMMENT '进度（0-100）',
273     `label_version` VARCHAR(100) DEFAULT NULL COMMENT '导出时使用的标签版本快照',
274     `requested_by` BIGINT UNSIGNED NOT NULL COMMENT '请求导出用户ID（外键→sys_user.id）',
275     `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '请求时间',
276     `completed_at` DATETIME DEFAULT NULL COMMENT '完成（或失败）时间',
277     PRIMARY KEY (`id`),
278     KEY `idx_task` (`task_id`),
279     KEY `idx_status` (`status`),
280     CONSTRAINT `fk_export_task` FOREIGN KEY (`task_id`) REFERENCES `task` (`id`) ON DELETE CASCADE,
281     CONSTRAINT `fk_export_user` FOREIGN KEY (`requested_by`) REFERENCES `sys_user` (`id`) ON DELETE RESTRICT
282 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='导出日志表';
283
284 -- 导出文件明细表
285 CREATE TABLE `export_file` (
286     `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '导出文件明细ID（主键）',
287     `export_log_id` BIGINT UNSIGNED NOT NULL COMMENT '所属导出日志ID（外键→export_log.id）',
288     `file_url` VARCHAR(500) NOT NULL COMMENT '文件访问URL或服务器相对路径',
289     `expiry_at` DATETIME DEFAULT NULL COMMENT '临时文件过期时间',
290     PRIMARY KEY (`id`),
291     KEY `idx_log` (`export_log_id`),
292     CONSTRAINT `fk_efile_log` FOREIGN KEY (`export_log_id`) REFERENCES `export_log` (`id`) ON DELETE CASCADE
293 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='导出文件明细表';
```

索引设计表

表名	索引名称	索引字段	索引类型	设计原因
sys_user	PRIMARY	id	主键	唯一标识每条记录
sys_user	uk_username	username	唯一索引	登录时根据用户名查询，要求唯一且快速定位
sys_user	idx_status	status	普通索引	用户管理页面按状态筛选（正常/禁用）
sys_user	idx_role	role_id	普通索引	按角色查询用户（如列出所有任务创建者）

表名	索引名称	索引字段	索引类型	设计原因
sys_role	PRIMARY	id	主键	唯一标识每条记录
sys_role	uk_role_code	role_code	唯一索引	角色代码用于权限判断，要求唯一且快速查找
guide_version	PRIMARY	id	主键	唯一标识每条记录
guide_version	idx_active	is_active	普通索引	获取当前启用的指南版本 (is_active=1)
label_l1	PRIMARY	id	主键	唯一标识每条记录
label_l1	uk_version_abbr	(guide_version_id, abbr)	唯一复合索引	保证同一版本下一级标签简称唯一；支持按版本快速加载所有标签
label_l2	PRIMARY	id	主键	唯一标识每条记录
label_l2	idx_parent	parent_l1_id	普通索引	查询某个GM标签下的所有二级标签
relation_type	PRIMARY	id	主键	唯一标识每条记录
relation_type	uk_version_abbr	(guide_version_id, abbr)	唯一复合索引	保证关系类型简称唯一；支持按版本快速加载
task	PRIMARY	id	主键	唯一标识每条记录
task	idx_creator	creator_id	普通索引	查询某个创建者创建的所有任务

表名	索引名称	索引字段	索引类型	设计原因
task	idx_status	status	普通索引	任务列表按状态筛选（待标注/待裁定/可导出）
task	idx_guide_version	guide_version_id	普通索引	查询使用某个指南版本的所有任务（统计/分析）
global_document	PRIMARY	id	主键	唯一标识每条记录
global_document	idx_uploader	uploaded_by_id	普通索引	查询某个上传者上传的所有全局文件
global_document	idx_file_name	file_name(100)	普通索引	按文件名模糊搜索（前缀匹配）
task_document	PRIMARY	id	主键	唯一标识每条记录
task_document	idx_task	task_id	普通索引	核心索引： 查询某个任务下的所有数据条目
task_document	idx_global	global_doc_id	普通索引	查询某个全局文件被哪些任务引用
task_assignment	PRIMARY	id	主键	唯一标识每条记录
task_assignment	uk_task_doc_user_role	(task_id, document_id, user_id, role_in_task)	唯一复合索引	防止重复分配；同时覆盖查询“某任务某文档的所有标注员”
task_assignment	idx_user	user_id	普通索引	查询用户参与的所有任务（P5页面）

表名	索引名称	索引字段	索引类型	设计原因
task_assignment	idx_task_role	(task_id, role_in_task)	复合索引	查询某任务的所有标注员（或所有裁决者）
proposition	PRIMARY	id	主键	唯一标识每条记录
proposition	idx_document	document_id	普通索引	查询某个文档的所有命题（标注工作台加载）
proposition	idx_sequence	(document_id, sequence_no)	复合索引	支持删除命题后重排序号：WHERE document_id=? AND sequence_no>?
relation	PRIMARY	id	主键	唯一标识每条记录
relation	idx_document	document_id	普通索引	查询某个文档的所有关系
relation	idx_parent	parent_relation_id	普通索引	递归查询嵌套关系树
relation_member	PRIMARY	id	主键	唯一标识每条记录
relation_member	idx_relation	relation_id	普通索引	查询某个关系的所有成员
relation_member	idx_member	(member_type, member_id)	复合索引	反向查询 ：查找哪些关系引用了某个命题或关系（删除前检查）
annotation	PRIMARY	id	主键	唯一标识每条记录

表名	索引名称	索引字段	索引类型	设计原因
annotation	uk_doc_user	(document_id, user_id)	唯一复合索引	保证每个标注员对每个文档仅一条有效结果；支持按文档和用户快速定位
annotation	idx_task	task_id	普通索引	冗余优化： 按任务批量加载所有标注结果（裁决比对）
annotation	idx_submitted	submitted_at	普通索引	按提交时间排序或统计标注进度
label_config	PRIMARY	id	主键	唯一标识每条记录
label_config	idx_task	task_id	普通索引	查询某个任务的配置历史
arbitration	PRIMARY	id	主键	唯一标识每条记录
arbitration	uk_document	document_id	唯一索引	确保每个文档仅一条最终裁决结果
arbitration	idx_task	task_id	普通索引	冗余优化： 按任务批量获取所有裁决结果（导出）
arbitration	idx_arbitrator	arbitrator_id	普通索引	查询某个裁决者处理过的所有文档
export_log	PRIMARY	id	主键	唯一标识每条记录
export_log	idx_task	task_id	普通索引	查询某个任务的导出历史

表名	索引名称	索引字段	索引类型	设计原因
export_log	idx_status	status	普通索引	异步导出任务轮询：获取待处理或处理中的导出请求
export_file	PRIMARY	id	主键	唯一标识每条记录
export_file	idx_log	export_log_id	普通索引	查询某个导出请求的所有文件

3.3 AI辅助审查

一、第三范式（3NF）符合性审查

结论：整体设计高度符合第三范式，结构规范、冗余度低

符合点

- 1. 所有表均有独立主键，无重复数据行
- 2. 字段完全依赖主键，无部分依赖 / 传递依赖
- 3. 多对一、一对多关系全部通过外键实现，无数据冗余
- 4. 业务拆分清晰：用户、角色、指南、任务、文档、标注、裁决、导出完全解耦
- 5. 关联关系标准化：如 `relation_member` 用多态关联替代冗余字段，符合范式设计

二、索引设计合理性审查

结论：索引设计完整、精准、无冗余，完全满足业务查询需求

优秀设计点

- 1. 主键索引全覆盖，所有表使用自增 ID 做主键
- 2. 唯一索引合理：用户名、角色代码、文档 - 用户标注关系等做唯一约束
- 3. 业务查询索引齐全
 - 用户表：username、role_id、status
 - 任务表：task_id、status、creator_id
 - 标注 / 裁决 / 关系表：document_id、task_id、user_id
 - 指南配置表：version_id
- 4. 联合索引精准：如 `(document_id, sequence_no)` 适配排序查询
- 5. 外键全部建索引：避免联表查询性能灾难

三、潜在性能瓶颈审查

结论：无严重性能风险，仅存在少量可预见瓶颈，优化后可支撑高并发使用

潜在瓶颈

1. 大文本字段存储风险

- `extracted_text`、`selected_text` 为 LONGTEXT/TEXT
- 风险：查询时拖慢速度、占用大量内存

2. JSON 字段无法索引，复杂查询慢

- `annotation`、`label_config`、`arbitration` 大量使用 JSON
- 风险：无法对 JSON 内部字段检索

3. 联表层级较深

- 标注→文档→任务→指南→标签 五级关联
- 风险：复杂报表查询慢

4. 无分表策略

- `proposition`、`relation` 数据量可能千万级
- 风险：单表过大